

分类号: _____

单位代码: 10335

密 级: _____

学 号: _____

浙江大学

工程硕士专业学位论文



中文论文题目: 基于图优化的移动机器人 SLAM 建图算法研究

英文论文题目: Research on SLAM mapping algorithm of mobile robot based on graph optimization

申请人姓名: 王阳

指导教师: 徐月同 副教授

合作导师: 徐冠华 高工

专业学位类别: 工程硕士

专业学位领域: 机械工程

所在学院: 工程师学院

论文提交日期 2022 年 7 月 15 日

基于图优化的移动机器人 SLAM 建图算法研究



论文作者签名: 王阳

指导教师签名: 何何何

论文评阅人 1: 隐名评阅

评阅人 2: 隐名评阅

评阅人 3: 隐名评阅

评阅人 4: _____

评阅人 5: _____

答辩委员会主席: 王亚卡/正高级工程师/中建材智能自动化研究院

委员 1: 沈洪垚/教授/浙江大学

委员 2: 姚鑫骅/副教授/浙江大学

委员 3: 胡松钰/副教授/浙江大学

委员 4: _____

委员 5: _____

答辩日期: 2022 年 9 月 22 日

致谢

时光飞逝，在浙江大学的研究生学习生涯即将结束，回望过去，收获良多。值此论文完成之际，我要向曾经给予我支持、帮助、关心的师长、同学和亲友表达真诚的谢意。

首先，由衷感谢我的导师徐月同副教授，徐老师渊博的学识、严谨的科研态度、诲人不倦的精神让我受益匪浅。徐老师对研究方向的把控，让我得以避免多走弯路，毕业论文的选题、撰写、修改的整个过程都离不开徐老师的谆谆教导。此外，在徐老师的教导下，还学到了很多做人做事的道理，使我终生受益。

衷心感谢我的校外导师徐冠华博士，在科研与工作上对我的引领与指导，在学习与生活上对我的帮助与关心。徐博士认真踏实的工作态度、求是严谨的工作作风给我留下了深刻的印象，是我们学习的榜样。研究生期间的实践学习、科学研究以及论文的选题、撰写、修改，都离不开徐博士的悉心指导，对此表示真挚的感谢。

同时，感谢傅建中教授，傅老师才望高雅，平易近人，为我们提供了轻松良好的实验室学术氛围。感谢孙扬帆博士在工程实践中对我的帮助与指导，感谢王郑拓博士对论文研究内容的指导。

衷心的感谢师兄邓容新、阳贤文、乔梁、江谢木、董大钊、周彤、赵磊，同窗好友马江、李天辰、马家骏、刘茂霖、杨兴飞，师弟陆昱翔在研究生学习期间给与我的帮助。

深深感谢我的家人，感谢他们给与我的支持和鼓励，让我有了跌倒之后再爬起来的力量。

感谢浙江大学。让我明白了“求是创新”的真正的含义。感谢浙江大学提供的学习平台，让我接触到前沿的知识理论。这三年年改变了我的人生轨迹，让我懂得了人生的意义。

最后，衷心祝各位支持、帮助、关心过我的师长、同学、亲人幸福美满！

王阳

二〇二二年九月

摘要

随着移动机器人技术的快速发展，移动机器人广泛应用于企业生产，企业对移动机器人的性能有了更高的要求。同步定位与建图（Simultaneous Localization and Mapping, SLAM）技术可以帮助机器人实现自主导航，对提高移动机器人的性能有重要的作用。本文对基于图优化的激光 SLAM 算法进行了研究，具体的研究内容如下：

首先，设计了移动机器人平台的硬件系统和软件系统。提出了硬件系统的整体设计方案，对其中的控制模块、感知模块、机械模块和驱动模块进行选型和设计；提出了软件系统的整体框架，采用开源机器人操作系统 ROS 设计了上位机软件系统，并对下位机嵌入式软件系统的数据采集模块、运动控制模块进行了设计。

其次，研究了 SLAM 激光里程计算法，实现了移动机器人的位姿估计和子图构建。采用 IMU 数据对激光点云进行畸变矫正；使用差速驱动机器人的位姿估计算法对机器人位姿进行估计，并为帧与图扫描匹配算法提供初值；通过启发式方法实现了关键帧的选取，根据选取的关键帧进行子图构建；采用 Magazino 数据集，分别验证了激光里程计和差速驱动机器人位姿估计算法的有效性。

然后，研究了 SLAM 后端优化算法，减小了前端激光里程计的累计误差，提高了算法的建图性能。通过像素匹配进行闭环检测，使用分支定界加速闭环检测，提高了闭环检测的速度；根据检测到的闭环构建位姿图，建立目标函数，并使用列文伯格-马夸尔特法进行求解，实现了位姿图优化；采用 Magazino 数据集，验证了后端优化算法的有效性；基于德意志博物馆的数据集，验证了闭环检测算法的实时性。

最后，搭建了实验平台并开展了实验研究。搭建了双轮差速移动机器人实验样机，在室内环境开展了建图实验，通过比较本文激光里程计算法与 Hector 算法的建图效果、建图精度，验证了本文激光里程计算法的有效性；在室内环境和长廊环境分别开展了建图实验，通过比较本文 SLAM 算法与 Cartographer 算法的建图效果、建图精度以及算法实时性，验证了本文 SLAM 算法的有效性。

关键词：同步定位与建图；图优化；激光里程计；后端优化；

Abstract

With the rapid development of mobile robot technology, mobile robot is widely used in enterprise production, and enterprises have higher requirements for the performance of mobile robot. Synchronous Location and Mapping (SLAM) technology can help the robot to achieve autonomous navigation and plays an important role in improving the performance of the mobile robot. In this paper, the laser SLAM algorithm based on graph optimization is studied. The specific research contents are as follows:

Firstly, the hardware system and software system of the mobile robot platform are designed. The overall design scheme of the hardware system is proposed, and the control module, sensing module, mechanical module and drive module are selected and designed; The overall framework of the software system is proposed. The upper computer software system is designed by using the open source robot operating system ROS, and the data acquisition module and motion control module of the lower computer embedded software system are designed.

Secondly, the SLAM laser mileage calculation method is studied, and the pose estimation and submap construction of the mobile robot are realized. The IMU data is used to correct the distortion of the laser point cloud; the pose estimation algorithm of the differential driven robot is used to estimate the pose of the robot, and the initial value is provided for the frame and graph scanning matching algorithm; the key frames are selected by the heuristic method, and the subimages are constructed according to the selected key frames; the effectiveness of the laser odometer and differential driven robot pose estimation algorithms are verified by using Magazino data sets.

Then, the SLAM back-end optimization algorithm is studied, which reduces the cumulative error of the front-end laser odometer and improves the mapping performance of the algorithm. The closed-loop detection is carried out by pixel matching, and the branch and bound is used to accelerate the closed-loop detection, which improves the speed of closed-loop detection. According to the detected closed-loop position and attitude map, the objective function is established, and the Lewenberg-Marquart method is used to solve the problem. The Magazino data set is used to verify the effectiveness of the back-end optimization algorithm. Based on the

data set of the German Museum, the real-time performance of the closed-loop detection algorithm is verified.

Finally, the experimental platform is built and the experimental research is carried out. An experimental prototype of a two-wheeled differential mobile robot is built, and the mapping experiment is carried out in the indoor environment. By comparing the mapping effect and accuracy of the laser mileage calculation method and the Hector algorithm, the effectiveness of the laser mileage calculation method is verified. Mapping experiments are carried out in indoor environment and corridor environment respectively. By comparing the mapping effect, accuracy and real-time performance of the proposed SLAM algorithm and the Cartographer algorithm, the effectiveness of the SLAM algorithm is verified.

Keywords: Simultaneous Localization and Mapping; map optimization; laser odometer; back-end optimization

目次

致谢I

摘要 II

Abstract III

目次 V

图目录 VIII

表目录 XI

1 绪论 1

 1.1 研究背景及意义 1

 1.2 研究现状 2

 1.2.1 移动机器人发展现状 2

 1.2.2 激光 SLAM 算法研究现状 3

 1.2.3 基于图优化激光 SLAM 算法研究现状 7

 1.2.4 激光 SLAM 算法研究现状评述 11

 1.3 论文研究内容和章节安排 12

 1.4 本章小结 13

2 移动机器人平台设计 14

 2.1 移动机器人的硬件系统 14

 2.1.1 整体方案 14

 2.1.2 控制模块 15

 2.1.3 感知模块 15

 2.1.4 机械模块 17

 2.1.5 驱动模块 17

 2.2 移动机器人的软件系统 18

 2.2.1 ROS 简介 18

 2.2.2 软件系统整体架构 19

 2.2.3 上位机软件系统 20

 2.2.4 下位机软件系统 22

2.3 本章小结	23
3 激光里程计算法研究	24
3.1 算法框架	24
3.2 激光点云畸变矫正算法研究	25
3.3 激光点云的扫描匹配算法研究	27
3.3.1 帧与图的扫描匹配算法	27
3.3.2 差速驱动机器人位姿估计算法	30
3.4 子图更新	32
3.4.1 关键帧选取	32
3.4.2 栅格地图构建	33
3.5 仿真实验	34
3.5.1 评价标准	34
3.5.2 激光里程计实验	35
3.5.3 位姿估计算法实验	37
3.6 本章小结	38
4 后端优化算法研究	39
4.1 算法框架	39
4.2 闭环检测算法	40
4.2.1 基于像素匹配的闭环检测	40
4.2.2 基于分支定界的闭环检测	42
4.3 位姿图优化算法	44
4.4 仿真实验	46
4.4.1 后端优化算法实验	46
4.4.2 闭环检测算法实验	47
4.5 本章小结	51
5 实验与分析	52
5.1 实验平台搭建	52
5.2 激光里程计的对比实验	53
5.2.1 实验方案	53

5.2.2 实验结果与分析 55

5.3 SLAM 算法的对比实验 56

5.3.1 实验方案 56

5.3.2 实验结果与分析 57

5.4 本章小结 63

6 总结与展望 64

6.1 全文总结 64

6.2 工作展望 65

参考文献 66

图目录

图 1.1 负载机器人 1

图 1.2 欧美部分企业的移动机器人产品 2

图 1.3 移动机器人典型应用 2

图 1.4 国内企业的移动机器人产品 3

图 1.5 基于 RBPF 的 SLAM 算法 4

图 1.6 Gmapping 算法 5

图 1.7 基于图优化的 SLAM 算法框架 5

图 1.8 Kart SLAM 算法 6

图 1.9 Cartographer 算法 6

图 1.10 Cartographer 算法框架 7

图 1.11 ICP 算法流程 8

图 1.12 基于特征的扫描匹配算法流程 8

图 1.13 论文章节安排 12

图 2.1 移动机器人硬件框架 14

图 2.2 精灵 P10 的工控机 15

图 2.3 星秒 LS-20H 16

图 2.4 深迪 IMU445 16

图 2.5 移动机器人三维模型结构图 17

图 2.6 驱动轮 18

图 2.7 话题通信方式示意图 19

图 2.8 TF 树示意图 19

图 2.9 软件系统整体架构 20

图 2.10 上位机软件系统整体架构 20

图 2.11 传感器采集模块流程 21

图 2.12 运动控制功能流程 21

图 2.13 激光 SLAM 系统框架 21

图 2.14 传感器数据采集功能流程 22

图 2.15 电机控制功能流程	22
图 3.1 激光里程计算法框架	24
图 3.2 IMU 和激光雷达时间戳示意图	26
图 3.3 激光点云畸变矫正算法流程图	27
图 3.4 激光点云畸变矫正前后对比图	27
图 3.5 双三次插值示意图	28
图 3.6 差速驱动机器人的运动模型	30
图 3.7 差速驱动机器人的三种运动状态	30
图 3.8 位姿估计算法流程	31
图 3.9 激光里程计效果图	32
图 3.10 栅格地图	33
图 3.11 激光点云观测模型	34
图 3.12 数据集的详细信息	35
图 3.13 本文激光里程计算法构建的地图	36
图 3.14 Hector 算法构建的地图	36
图 3.15 不同算法估计轨迹对比	36
图 3.16 对照激光里程计算法构建的地图	37
图 3.17 不同算法估计轨迹对比	37
图 4.1 激光 SLAM 的后端优化算法框架	39
图 4.2 暴力搜索流程	41
图 4.3 闭环约束示意图	44
图 4.4 数据集的详细信息	46
图 4.5 本文激光里程计算法构建的地图	46
图 4.6 本文 SLAM 算法构建的地图	46
图 4.7 不同算法估计轨迹对比	47
图 4.8 对照 SLAM 算法构建的地图	48
图 4.9 不同 SLAM 算法 CPU 使用率对比	48
图 4.10 德意志博物馆数据集	49
图 4.11 本文 SLAM 算法构建的地图	49

图 4.12 对照 SLAM 算法构建的地图	50
图 4.13 不同 SLAM 算法 CPU 使用率对比	50
图 5.1 移动机器人样机	52
图 5.2 SLAM 系统地图显示界面	53
图 5.3 运动控制界面	53
图 5.4 实验室的室内场景	54
图 5.5 激光雷达的参数配置	54
图 5.6 数据集录制	54
图 5.7 本文激光里程计算法构建的地图	55
图 5.8 Hector 算法构建的地图	55
图 5.9 特征分布	55
图 5.10 不同算法的相对误差对比	56
图 5.11 实验室的室内环境	57
图 5.12 长廊环境	57
图 5.13 本文 SLAM 算法构建的地图	58
图 5.14 Cartographer 算法构建的地图	58
图 5.15 特征分布	58
图 5.16 不同 SLAM 算法的相对误差对比	59
图 5.17 不同 SLAM 算法 CPU 使用率对比	60
图 5.18 本文 SLAM 算法构建的长廊地图	60
图 5.19 Cartographer 算法构建的长廊地图	60
图 5.20 特征分布	61
图 5.21 不同 SLAM 算法的相对误差对比	62
图 5.22 不同 SLAM 算法 CPU 使用率对比	62

表目录

表 1.1 激光 SLAM 算法对比 7

表 1.2 扫描匹配方法的对比 9

表 1.3 闭环检测方法对比 10

表 2.1 激光雷达相关参数 16

表 2.2 IMU 的相关参数 16

表 2.3 驱动轮的主要参数 18

表 3.1 绝对轨迹误差 36

表 3.2 绝对轨迹误差 38

表 4.1 绝对轨迹误差 47

表 5.1 本文激光里程计算法的测量数据 56

表 5.2 Hector 算法的测量数据 56

表 5.3 本文 SLAM 算法的测量数据 59

表 5.4 Cartographer 算法的测量数据 59

表 5.5 本文 SLAM 算法的测量数据 61

表 5.6 Cartographer 算法的测量数据 61

1 绪论

本章首先介绍了激光 SLAM 算法的研究背景和研究意义，对移动机器人的发展现状进行了总结，然后对激光 SLAM 算法的研究现状进行了分析，着重对基于图优化的 SLAM 算法研究现状进行总结，并对激光 SLAM 算法存在的问题进行了评述，最后针对基于图优化的 SLAM 算法存在的问题，给出了本文的研究内容和章节安排。

1.1 研究背景及意义

随着计算机和传感器技术的发展，移动机器人技术取得了快速的发展，相关产品已经应用到工业生产、物流运输的方方面面，成为了提高人类生产效率和生活水平的重要工具。在“中国制造 2025”和“工业 4.0”的背景下，随着我国对中高端装备制造业关注程度的提升，为了提升制造效益，提高企业竞争力，工厂正朝着智能化、数字化的方向转型发展^[1]。在此背景下，移动机器人作为一种自动化设备，凭借其自由、灵活、高效、可靠的特点，被广泛应用于工厂生产中^[2]，当前工厂中使用的移动机器人有负载机器人、叉车机器人、码垛机器人等，负载机器人如图 1.1 所示。



图 1.1 负载机器人

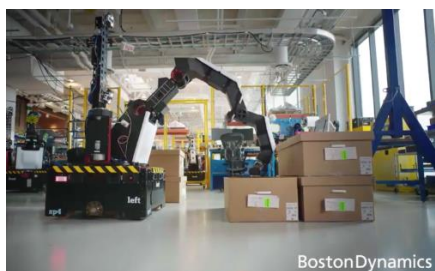
同步定位与建图（Simultaneous Localization and Mapping, SLAM）是实现移动机器人在未知环境下自我定位和地图构建的关键技术。SLAM 技术使用移动机器人搭载的各种传感器感知环境信息，建立合适的数学模型，估计机器人在环境中的相对位姿。根据传感器类型，SLAM 技术分为视觉 SLAM 和激光 SLAM 技术，由于视觉 SLAM 技术依赖成像器件，极易受到环境光的影响，在抗干扰和精度方面都逊色于激光 SLAM 技术，因此在实际应用中激光 SLAM 技术占主导地位^[3]。

目前激光 SLAM 算法已经发展比较成熟，并且基于激光 SLAM 的移动机器人已经应用在工业生产中。但是激光 SLAM 在实际应用中仍然存在定位精度不高、地图构建不精确的问题，这些问题给执行任务的移动机器人带来了一些障碍，所以有必要对激光 SLAM 算法展开更加深入的研究。

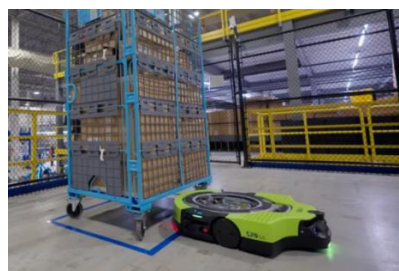
1.2 研究现状

1.2.1 移动机器人发展现状

世界第一台移动机器人是斯坦福大学制造的 Shakey，该机器人可以自动感知环境并进行路径规划，随着时代的发展，移动机器人的自动化、智能化水平也在不断提高。欧美等国家在移动机器人研发方面走在世界前列，国外移动机器人的代表企业有亚马逊、波士顿等，图 1.2 所示为欧美等国家将移动机器人用于工业生产，其中图 1.2 (a) 所示为波士顿动力的搬运机器人；其中图 1.2 (b) 所示为亚马逊的仓储机器人。



(a) 波士顿动力的搬运机器人



(b) 亚马逊的仓储机器人

图 1.2 欧美部分企业的移动机器人产品

我国对移动机器人的研究起步较晚，但是目前我国在移动机器人的各个方面有了较大突破，目前国内有一大批研发生产移动机器人的企业，如仙工、迈睿、海康威视、旷世等，这些企业很大程度上推动了我国移动机器人行业的发展，图 1.3 所示为移动机器人用于仓储物流、物料搬运。



(a) 移动机器人用于仓储物流



(b) 移动机器人用于物料搬运

图 1.3 移动机器人典型应用

随着智能化水平提高,移动机器人的性能也在不断提高,从开始的导引机器人,逐渐发展为自主移动机器人。图 1.4 为国内企业的移动机器人产品,其中图 1.4 (a) 所示为迈睿的潜伏系列机器人,可以进行自主导航和路径规划,定位精度达到 $\pm 5\text{ mm}$;图 1.4 (b) 所示为仙工的 AMB-800K,该机器人使用二维码和激光 SLAM 导航,导航精度达到 $\pm 5\text{ mm}$ 。



(a) 迈睿潜伏系列机器人



(b) 仙工顶升机器人

图 1.4 国内企业的移动机器人产品

1.2.2 激光 SLAM 算法研究现状

激光 SLAM 算法根据激光雷达不同分为 2D 激光 SLAM 算法和 3D 激光 SLAM 算法,3D 激光雷达具备测距精度高、抗光照、角度变化干扰性强的特性,适合室外大型环境地图的构建;2D 激光雷达的价格低廉,2D 激光 SLAM 算法可以保证室内环境下的建图精度,所以在移动机器人中应用广泛。基于 2D 激光 SLAM 的移动机器人已经应用于工业场景。

多传感器融合相比于纯激光算法具有巨大的性能优势,纯激光雷达需要花费巨大代价才能改善的问题,通过多传感器融合大部分都可以很好的解决,SLAM 融合技术主要分为两种主流的方法:松耦合和紧耦合,不同融合技术的特点相差甚远。松耦合方法是分别估计激光雷达点云位姿和 IMU 数据帧的相对运动,并且融合该两种数据的估计结果,从而获得移动机器人的位姿信息。紧耦合是通过使用估计器对激光雷达信和 IMU 数据融合得到最优估计。紧耦合相较于松耦合获得结果精度更高,稳健性更好,但是计算复杂度更高,实现要求更高,如数据实时同步。

根据激光 SLAM 算法的理论基础不同,激光 SLAM 算法可以分为两大类:一类是基于滤波的激光 SLAM 算法,一类是基于图优化的 SLAM 算法。基于滤波的 SLAM 算法可以分为:基于卡尔曼滤波的 SLAM 算法和基于粒子滤波的 SLAM 算法。

Smith 等^[4]首次提出了 SLAM 的概念，同时使用卡尔曼滤波算法对机器人的位姿和环境地图进行估计，该算法假定机器人的状态噪声和传感器的观测噪声服从高斯分布，根据系统输入的观测数据对机器人的状态和地图进行估计；Durrant-Whyte 等^[5]针对实际环境比较复杂，不符合简单线性模型的问题，提出了基于扩展卡尔曼滤波的 SLAM 算法，该算法采用一阶泰勒公式将非线性的机器人运动模型和观测模型线性化，在此基础上 Moutarlier 等^[6]实现了完整的 SLAM 算法框架。Julier 等^[7]提出基于无迹卡尔曼滤波的 SLAM 算法，该算法在模型的线性化方面有更高的精度；Jiang 等^[8]将自适应卡尔曼滤波应用到 SLAM 算法中，该算法通过实时调整卡尔曼滤波的参数，有效地提高了滤波效率，进而提高了 SLAM 算法定位的准确性。

此外，基于卡尔曼滤波的 SLAM 算法由于原理特性，需要大量的计算资源，为了降低该算法的计算量，Tardós 等^[9]提出了基于分割原理的更新方法；Folkesson 等^[10]提出了基于稀疏信息的处理方法；Bosse 等^[11]提出先构建子地图再建图的方法。

Hammersley 等^[12]首次提出了粒子滤波的思想，Del Moral 等^[13]首先将粒子滤波的思想应用到中 SLAM 算法中，为了降低该方法的计算复杂度，Murphy 等^[14]结合粒子滤波和 Rao-Blackwellized 的思想，提出了基于 RBPF 的 SLAM 方法，该方法的流程如图 1.5 所示，首先从上一时刻的粒子中进行采样获取当前时刻粒子的先验分布集合；然后计算每个粒子的权重，根据粒子的权重进行重采样；最后每个粒子根据传感器的观测数据和机器人的位姿，更新地图。在此基础上，Beevers^[15]和 Brunskill^[16]实现了基于 RBPF 的 SLAM 系统。

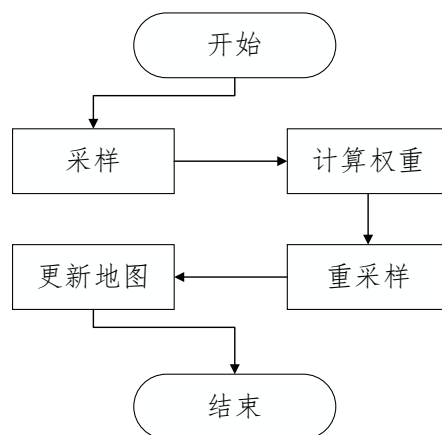


图 1.5 基于 RBPF 的 SLAM 算法

Montemerlo 等^[17]提出了 FastSLAM1.0，该方法使用 RPBF 方法估计机器人的位置，使用扩展卡尔曼滤波的方法估计路标的位置；之后，Montemerlo 等^[18]又提出了

FastSLAM2.0; 通过进一步对 FastSLAM 进行优化, Grisetti^[19]等提出了 Gmapping 算法, 该算法通过改进重采样, 改善了粒子退化的情况, 该算法的建图效果如图 1.6 所示; Liao 等^[20]利用 Gmapping 进行室内地图的构建, 证明了算法的鲁棒性。



图 1.6 Gmapping 算法

基于滤波的激光 SLAM 方法, 在小范围场景中具有不错的建图效果, 由于只考虑机器人当前时刻的位姿状态和观测信息, 当机器人状态出现偏差时, 偏差无法消除; 而且随着建图场景的增大, 消耗的计算资源逐步增加; 该方法不能用于构建大场景的地图^[21,22]。

基于图优化的 SLAM 算法由 Lu 等^[23]提出, 该算法认为机器人的状态与之前所有时刻的观测数据有关, 利用图论对 SLAM 问题进行建模, 图是一种数据结构, 由顶点和边组成。Golfarelli 等^[24]将基于图优化的 SLAM 系统认为是质量弹簧模型系统, 其中图中的节点用带质量的节点表示, 节点与节点之间的约束用弹簧表示, 弹簧的刚度即约束的协方差。Gutmann 等^[25]提出基于图优化的激光 SLAM 算法框架, 该算法框架如图 1.7 所示, 框架包含前端和后端两部分, 其中前端进行扫描匹配和闭环检测, 后端进行后端优化。

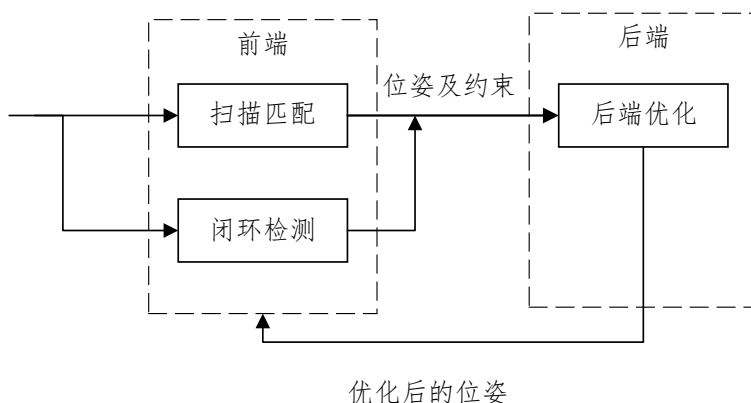


图 1.7 基于图优化的 SLAM 算法框架

基于图优化的 SLAM 算法计算量大, 受限当时的研究水平, 无法满足算法实时性需求, 随着求解方法不断更新和发展, 在相同计算量条件下基于图优化的 SLAM 求解速度已远超基于滤波的 SLAM 方法。其中 Konolige 等^[26]认识到了系统的稀疏性, 提出了基于图优化框架的开源 SLAM 算法 Karto SLAM, 该算法的建图效果如图 1.8 所示; 此外, Hess 等^[27]提出的 Cartographer 算法是基于图优化的 SLAM 算法, 该算法的建图效果如图 1.9 所示。



图 1.8 Karto SLAM 算法

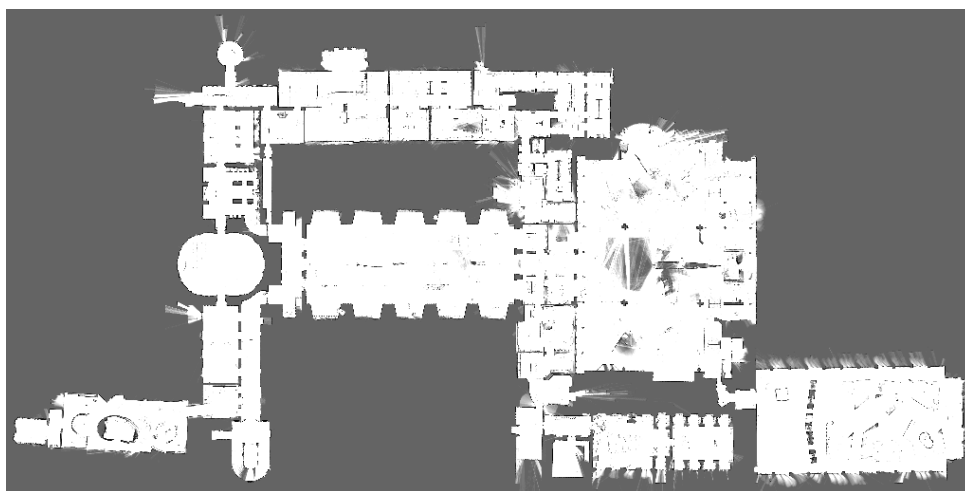


图 1.9 Cartographer 算法

图 1.10 是 Cartographer 算法框架, 该算法主要由局部 SLAM 和全局 SLAM 两部分组成, 局部 SLAM 部分采用扫描匹配估计机器人的位姿, 采用运动滤波进行关键帧选取, 根据关键帧进行子图构建; 全局 SLAM 部分首先进行闭环检测, 根据检测的闭环进行约束计算, 然后通过后端优化算法得到优化后的位姿, 最后输出优化后的地图。在实际应用中, Cartographer 算法对机器人硬件要求不高, 提高机器人的硬件水平, 会改善 Cartographer 算法的建图性能, Zhu 等^[28]将工业级计算机作为机器人的核心控制器, 使用该算法在室内建图, 能够高效的完成了建图任务。

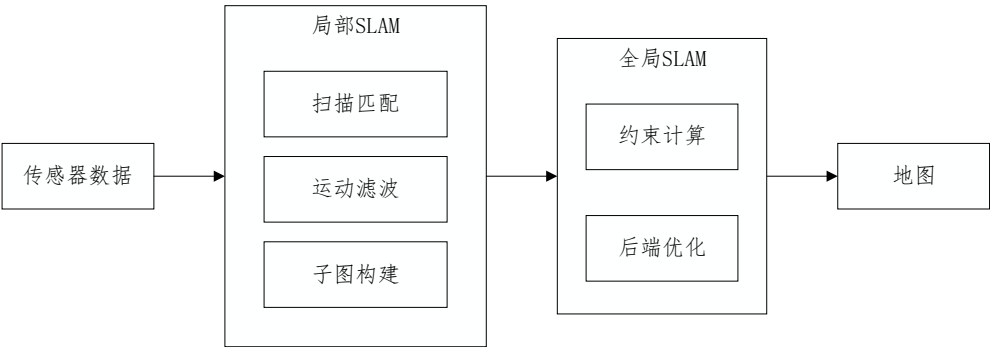


图 1.10 Cartographer 算法框架

表 1.1 为激光 SLAM 算法的对比，其中基于滤波的激光 SLAM 算法在大场景中环境中建图效果表现较差，基于图优化的激光 SLAM 算法在小场景环境和大场景环境中建图效果较好，因此基于图优化的激光 SLAM 算法是目前的研究重点。

表 1.1 激光 SLAM 算法对比

算法	基于滤波的激光 SLAM 算法	基于图优化的激光 SLAM 算法
小场景建图	较好	好
大场景建图	较差	好

1.2.3 基于图优化激光 SLAM 算法研究现状

基于图优化的激光 SLAM 算法主要由扫描匹配、闭环检测、后端优化等三部分组成。扫描匹配是 SLAM 算法的基础，现有的扫描匹配方法有：基于点的扫描匹配、基于特征的扫描匹配、基于数学特征的扫描匹配、基于优化的扫描匹配。

基于点的扫描匹配算法直接对相邻两帧激光点云进行匹配，1992 年，Chen^[29]提出了迭代最近点算法（ICP），该算法流程如图 1.11 所示，其通过计算相邻两帧激光点云的欧式距离来计算相对位姿变换。为了提高匹配精度和收敛速度，Minguez^[30]提出了 MbICP 算法，该算法重新定义了距离度量，同时考虑平移和旋转；Censi^[31]提出了 PLICP 算法，该算法利用点线距离来实现 ICP；Segal^[32]提出了 GICP 算法，该算法考虑了点周围的表面形状。除此之外，Hong^[33]指出在扫描匹配前需要对激光点云进行畸变校正；Yang^[34]将 ICP 与分支定界法结合，来保证解的全局最优。基于点的扫描匹配算法的缺点有：容易在局部发生收敛，在单一化的场景中容易出现匹配错误，没有考虑传感器存在的噪声等。

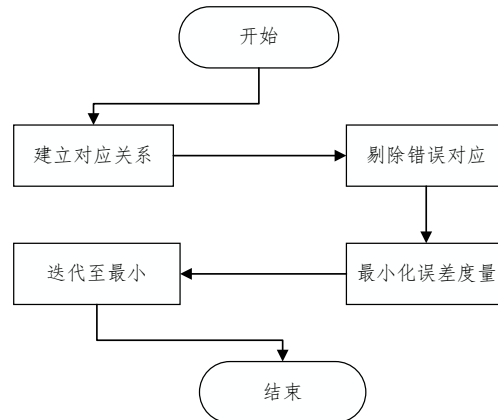


图 1.11 ICP 算法流程

基于特征的扫描匹配算法需要从激光点云数据中提取点、线、面等元素进行匹配，该算法的流程如图 1.12，首先对激光点云进行预处理，然后从激光点云中提取场景特征，进行特征筛选，最后进行特征匹配并输出匹配结果。Jensfelt 等^[35]从激光点云数据中提取边角点和拐角点用于扫描匹配；Nakamura 等^[36]从激光点云数据中提取交叉点、端点、角点用于扫描匹配；Mohamed 等^[37]从激光点云数据中提取线段之间的交叉点作为角点，基于角点进行扫描匹配；Shifei 等^[38]提出的分割-合并方法是 2D 扫描匹配中常用线段分割方法；Siadat 等^[39]针对 2D 扫描匹配提出了的 3 种线段分割方法，分别是边缘跟随、线段跟踪、迭代端点拟合；Núñez 等^[40]引入曲率方法，使用自适应曲率函数将激光点云数据区分为曲线和直线。基于特征的扫描匹配算法的缺点有在非结构化的场景难以提取稳定可靠的特征，提取特征的计算代价较高，匹配的精度较差。

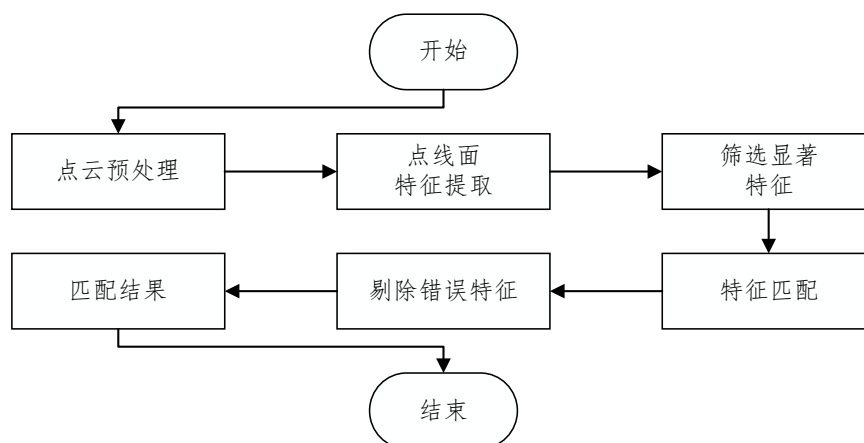


图 1.12 基于特征的扫描匹配算法流程

基于数学特征的扫描匹配算法是利用数学性质来刻画激光点云数据和位姿变换的算法，其中最常用的是基于正态分布变换的方法。Biber 等^[41]首先提出了基于正态分布变换的扫描匹配算法，该算法不计算点与点的变换关系，而是把离散的激光点变换为概率密

度，概率密度呈正态分布，计算当前激光点云数据在概率密度上的最大得分。该算法是 2D 扫描匹配中较为成熟的解决方法，具有较高的效率和较快的收敛速度，但是该算法对初值比较敏感，容易陷入局部最优。

基于优化方法的扫描匹配方法，就是将扫描匹配问题转化为非线性最小二乘优化问题，该方法对初值比较敏感，匹配效果取很大程度上决于初值。该方法是目前主流的扫描匹配方法，Hector^[42]和 Cartographer^[27]均使用基于优化方法的扫描匹配方法。

表 1.2 为扫描匹配方法的对比，其中基于点的扫描匹配方法运行速度慢，基于特征的扫描匹配方法和基于数学特征的扫描匹配方法定位精度较差，基于优化方法的扫描匹配方法定位精度高、运行速度快、应用范围广，因此基于图优化的激光 SLAM 算法一般采用基于优化方法的扫描匹配方法。

表 1.2 扫描匹配方法的对比

方法	精度	运行速度	应用范围
基于点的扫描匹配	高	慢	广
基于特征的扫描匹配	低	快	结构化场景
基于数学特征的扫描匹配	一般	较快	广
基于优化方法的扫描匹配	高	快	广

闭环检测是 SLAM 算法的重要组成，闭环检测的任务是实现全局数据关联，常用的闭环检测算法有有帧与帧的闭环检测方法、帧与子图的闭环检测方法、子图与子图的闭环检测方法。

帧与帧的闭环检测方法利用 Olson^[43]提出的相关性扫描匹配方法，将经过相对平移和旋转的激光点云数据进行匹配，因为单帧激光点云数据包含的场景特征较少，该方法容易出现闭环检测失败的情况。

帧与子图的闭环检测方法将激光点云和子图进行匹配，子图由固定数量的激光点云帧组成，该方法计算量适中，闭环检测效果好。该方法是目前主流的闭环检测方法，Cartographer^[27]使用该方法进行闭环检测。

子图与子图的闭环检测方法将当前生成的子图与之前的子图进行匹配，解决了单帧激光点云包含特征少的问题，该方法计算量大，但是计算精度较高。文国成^[44]使用定位筛选和压缩表对该方法进行优化，加快了闭环检测速度。

除此之外，Olson 等^[45]提出的多分辨率的闭环检测方法，将激光点云和多分辨率的地图进行匹配；Yin 等^[46]提出了基于神经网络的闭环检测方法；Rusu 等^[47]根据提取的描述子进行闭环检测；梁潇^[48]使用视觉辅助闭环检测，提高了建图效果；邹谦^[49]使用二维码

辅助闭环检测，取得了较好的建图效果；徐晓苏^[50]将视觉与惯导结合，对闭环检测过程进行优化，提高了 SLAM 系统稳定性。目前，闭环检测的难点主要体现在：当在相似的结构化场景中，会加剧闭环检测的难度；随着时间的增加需要处理数据规模逐渐增加，会降低 SLAM 算法的实时性。

表 1.3 为常用的闭环检测方法的对比，其中帧与帧的闭环检测方法准确性较低，子图与子图的闭环检测方法运行速度很慢，帧与子图的闭环检测方法运行速度快，准确性高，因此基于图优化的激光 SLAM 算法中一般使用帧与子图的闭环检测方法。

表 1.3 闭环检测方法对比

方法	运行速度	准确性
帧与帧的闭环检测	快	低
帧与子图的闭环检测	较快	高
子图与子图的闭环检测	很慢	高

后端优化方法利用机器人的观测数据对机器人的位姿进行优化，基于图优化的 SLAM 后端优化方法有：基于最小二乘的优化方法、基于随机梯度下降法的优化方法、基于松弛技术的优化方法、流形优化等。

Lu 等人^[23]将 SLAM 优化问题转化为最小二乘问题，该问题常用的求解有 Gauss-Newton 法、列文伯格-马夸尔特法。在不考虑问题的稀疏性时，假设节点的个数为 n ，使用列文伯格-马夸尔特法求解问题的时间复杂度为 $O(n^3)$ ，不能满足算法的实时性要求^[51]。为了提高算法的实时性，Dellaert 等^[52]提出利用问题内在的稀疏结构特性，使用稀疏矩阵分解对该问题进行求解；Kaess 等^[53]将图建模和稀疏线性代数方法结合，提出了 iSAM 方法；基于 iSAM 方法，Kaess 等^[54]通过对节点重新排序和重新线性化，提出了改进后的 iSAM2 方法；Konolige 等^[26]提出根据图约束快速构造稀疏矩阵的 SPA (sparse pose adjustment) 方法。目前，基于最小二乘的优化方法是后端优化的常用方法，该方法的比较简单，计算效率较高。

Olson 等^[55]采用随机梯度下降法进行后端优化，该方法在每次迭代时随机选取图中的一条边作为约束并计算相应的梯度下降方向，在该方向上寻找目标函数的最优解。此外，Grisetti 等^[56]使用树结构来表示位姿间的关系，通过增量方式表示待求解的状态，提高了算法性能；Gao 等^[57]将随机梯度下降方法应用在磁序列 SLAM 中，提高了算法在处理大规模数据时的性能。基于随机梯度下降的优化方法需要合适的优化方向和优化步长，其中优化方向为梯度的负方向，合适的优化步长将提高该方法的效率。

Duckett 等^[58]采用松弛技术进行后端优化,基本思想是根据当前节点与相邻节点之间的位置关系和相关约束重新计算当前节点位姿。此外,Frese 等^[51]提出了多级松弛策略,使用多重网格方法对偏微分方程进行求解,提高了算法效率;Carlone 等^[59]提出基于测量的松弛技术,提高了算法的鲁棒性。该方法的优化效率较高,但是在求解精度方面低于其他优化方法,所以该方法的实际应用较少。

在欧氏空间中,机器人位姿旋转分量的估计值会出现奇异,导致结果发散,针对这种情况,Grisetti 等^[60]提出在流形空间中优化的方法,避免了旋转分量可能出现奇异的情况;Kümmerle 等^[61]提出了基于流形的图优化框架 g2o,提高了算法开发效率;Hertzberg 等^[62]将流形方法应用到传感器的标定问题中。

后端优化算法是基于图优化的 SLAM 算法的核心部分,后端优化算法的主要研究方向是在保证算法实时性的前提下,提高算法的精度。

1.2.4 激光 SLAM 算法研究现状评述

通过对激光 SLAM 算法的研究现状分析可知,激光 SLAM 算法主要分为基于滤波的激光 SLAM 算法和基于图优化的激光 SLAM 算法。其中基于滤波的激光 SLAM 算法,在大场景环境中建图效果较差,在实际工程应用较少,基于图优化的激光 SLAM 算法鲁棒性好、可以构建大场景的地图,是目前主流的研究方法,也是实际工程应用中最为广泛的,因此本文对基于图优化的激光 SLAM 算法进行研究。

通过分析基于图优化的激光 SLAM 算法研究可知,该算法扫描匹配部分采用基于优化方法的扫描匹配方法,闭环检测部分采用帧与子图的闭环检测方法,后端优化部分采用基于最小二乘的优化方法。但是基于图优化的 SLAM 算法还存在一些问题,例如当扫描匹配算法的初值出现错误时,扫描匹配算法会计算出错误的位姿;当扫描匹配计算的位姿出现错误时,后端优化使用错误的位姿进行优化,可能导致构建的地图与实际环境不一致;随着建图范围的增加,需要将传感器的观测信息和越来越多的场景进行匹配,闭环检测速度变慢,无法保证算法的实时性;基于上述问题,本文针对基于图优化的激光 SLAM 系统,开展前端激光里程计、后端优化算法研究,从而提高 SLAM 算法的建图性能。

1.3 论文研究内容和章节安排

通过对激光 SLAM 研究现状的总结，可知基于图优化的激光 SLAM 算法是当前的主要研究方向。本文搭建了基于图优化的激光 SLAM 算法，着重对 SLAM 算法中的前端激光里程计算法和后端优化算法进行研究。全文共分为六个章节，各个章节的研究内容和具体安排如图 1.13 所示。

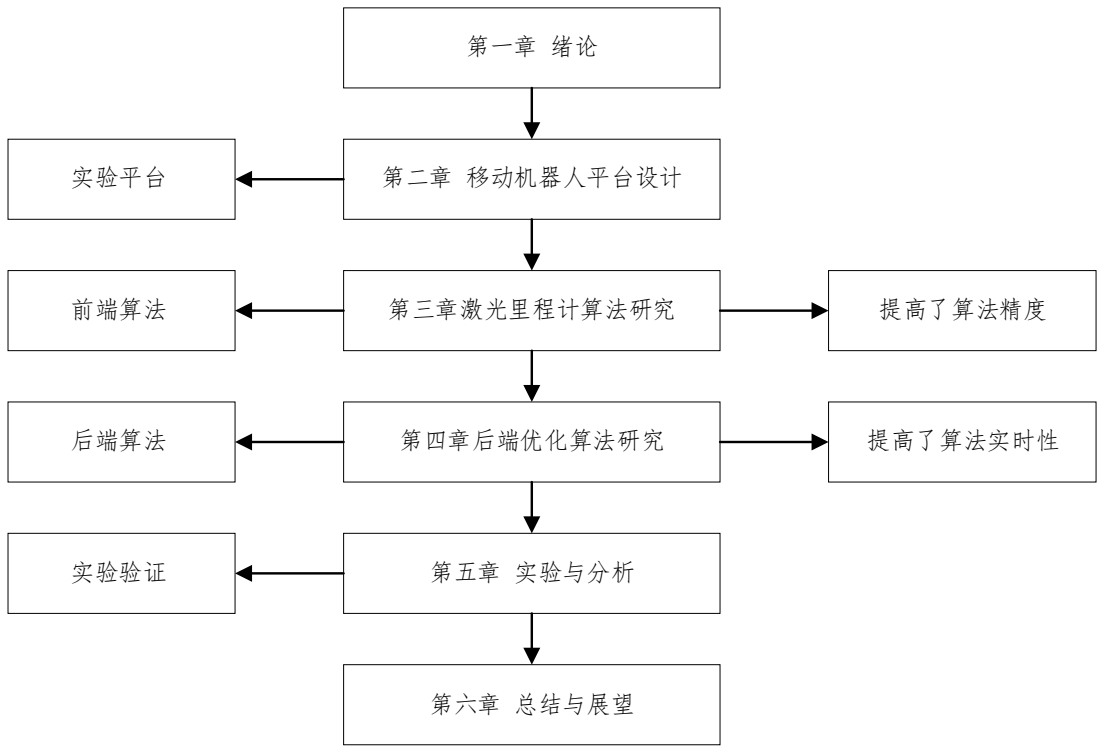


图 1.13 论文章节安排

第一章，绪论。介绍了激光 SLAM 算法的研究背景及意义，阐述了激光 SLAM 算法的研究现状，并给出了本文的研究内容和组织架构。

第二章，移动机器人平台设计。设计并搭建了移动机器人平台，给出了移动机器人的硬件系统组成，对硬件系统的控制模块、机械模块、驱动模块、感知模块进行了设计；接着设计了移动机器人的软件系统，基于 ROS 设计了上位机软件系统，并对下位机嵌入式软件系统的功能模块进行了设计。

第三章，激光里程计算法研究。研究并搭建激光里程计系统，激光里程计使用 IMU 数据对激光点云进行畸变矫正，使用差速驱动机器人的位姿估计算法对机器人位姿进行估计，并为帧与图的扫描匹配算法提供初值，通过启发式方法实现了关键帧的选取，并

根据选取的关键帧进行了子图构建。设计了激光里程计实验，验证了激光里程计的有效性；设计了位姿估计算法实验，验证了位姿估计算法的有效性。

第四章，后端优化算法研究。研究并搭建 SLAM 算法的后端优化模块，首先采用像素匹配进行闭环检测，使用分支定界加速闭环检测；然后根据检测到的闭环构建位姿图，建立目标函数；最后使用列文伯格-马夸尔特法进行目标函数求解。设计了后端优化算法实验，验证了后端优化算法的有效性；设计了闭环检测算法实验，验证了闭环检测算法的实时性。

第五章，实验与分析。进行了实验平台的搭建，通过比较本文激光里程计算法与 Hector 算法的建图效果、建图精度，验证了本文激光里程计算法的有效性；通过比较本文 SLAM 算法与 Cartographer 算法的建图效果、建图精度以及算法实时性，验证了本文 SLAM 算法的有效性。

第六章，总结与展望。对论文的研究内容进行了总结，指出了下一步研究方向。

1.4 本章小结

本章首先给出了激光 SLAM 算法的研究背景和意义，总结了机器人的发展现状，通过查阅文献对基于滤波的激光 SLAM 算法、基于图优化的激光 SLAM 算法研究现状进行总结。然后着重对基于图优化的激光 SLAM 算法进行介绍，对其中扫描匹配模块、闭环检测模块、后端优化模块的研究现状进行总结分析，指出了基于图优化的激光 SLAM 算法存在的问题。最后针对扫描匹配算法精度较低、SLAM 算法实时性较差的问题进行研究，给出了本文的研究内容和章节安排。

2 移动机器人平台设计

本章设计了移动机器人平台的硬件系统和软件系统，提出了硬件系统的整体设计方案，对其中的控制模块、感知模块、机械模块和驱动模块进行选型和设计；给出了软件系统的整体框架，采用开源机器人操作系统 ROS 设计了上位机软件系统，并对下位机嵌入式软件系统进行了设计。

2.1 移动机器人的硬件系统

2.1.1 整体方案

移动机器人平台的硬件系统主要由控制模块、感知模块、机械模块、驱动模块组成，移动机器人硬件框架如图 2.1 所示。其中控制模块由工控机和嵌入式控制器组成，工控机用于上位机软件系统的运行，嵌入式控制器用于下位机软件系统的运行；感知模块由内部传感器和外部传感器组成，内部传感器为 IMU 和编码器，外部传感器为激光雷达；机械模块由车身和安全防护装置组成，车身由底盘与车架组成，底盘用于驱动模块、控制模块、防撞装置等设备的安装，其中车架用于安装激光雷达、急停按钮与电源开关等设备，安全防护装置包括障碍报警器与具备缓冲作用的防撞条；驱动模块由驱动系统和电源组成，驱动系统由驱动轮、从动轮和减振机构组成，电源采用锂电池供电。

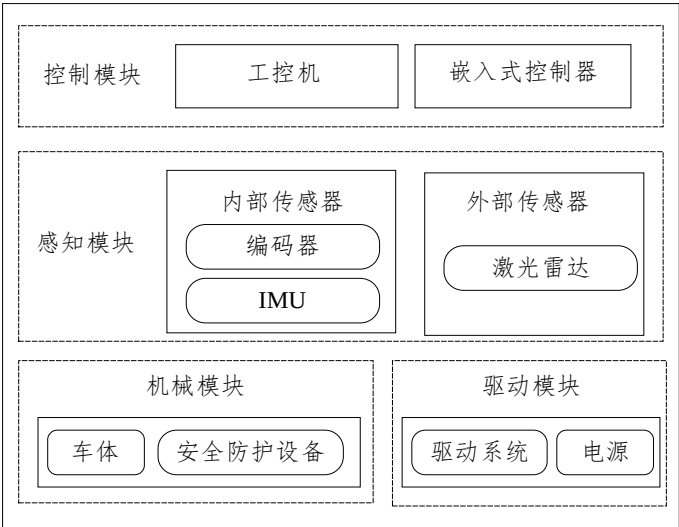


图 2.1 移动机器人硬件框架

2.1.2 控制模块

移动机器人的控制模块主要由工控机和嵌入式控制器组成，其中工控机主要用于上位机软件系统的运行，嵌入式控制器用于下位机软件系统的运行。工控机的主要功能有：通过串口与下位机进行通信；接收并处理下位机发送的传感器数据；根据运动命令，计算左右轮的转速，向下位机发送指令；进行同步定位与建图。本文选择上海稳信公司生产的精灵 P10，如图 2.2 所示，该工控机的使用 Intel 第四代 i5-4200U 2.6GHz 双核 CPU，搭载了 Ubuntu18.04 系统。



图 2.2 精灵 P10 的工控机

嵌入式控制器主要任务有：数据采集、运动控制、上位机通讯等，本文选择 STM32F103VET6 作为嵌入式控制器的主控芯片，该芯片有 512KB 的运行内存，运行频率为 72MHz，有 USB、CAN 和 USART 等多种通信方式，满足下位机软件系统的运行需求。

2.1.3 感知模块

激光雷达用于探测机器人周围的环境信息，目前按照技术路线有三角测距激光雷达和 TOF 激光雷达两种类型，其中三角测距激光雷达的精度低、采样率低，而 TOF 激光雷达的测量距离远、采样率高、精度高，因此本文选择 TOF 激光雷达。

TOF 激光雷达在发射激光脉冲时，记录脉冲的发射时间，在接收到反射回来的激光脉冲时，记录脉冲的返回时间，根据激光脉冲飞行时间计算激光雷达与反射物之间的距离。本文选择了星秒的 LS-20H 二维单线激光雷达，如图 2.3 所示，该激光雷达体积小方便安装。激光雷达的主要参数如表 2.1 所示。



图 2.3 星秒 LS-20H

表 2.1 激光雷达相关参数

参数项	参数值	参数项	参数值
扫描范围	0.1~20 m	扫描范围	270°
扫描精度	±30 mm	扫描频率	10-20 Hz
最小角度分辨率	0.08°	电源电压	DC12 V-24 V
工作温度	-10℃~55℃	数据传输接口	Ethernet 100BASE-TX
防护等级	IP65	功耗	2.5 W

IMU 是测量 x、y、z 轴的角速度与加速度的设备，本文选择的 IMU 是深迪半导体的 IMU445，如图 2.4 所示，IMU 尺寸为 24.1 mm × 37.7 mm × 10.8 mm。IMU445 是一个完整的惯性系统，包含完整的三轴陀螺仪和三轴加速度计，可以简单快速的集成到工业系统中，使用的常见的 SPI/UART 接口进行数据传输。IMU445 的陀螺仪和加速度计主要参数如表 2.2 表示。



图 2.4 深迪 IMU445

表 2.2 IMU 相关参数

	陀螺仪	加速度计
量程	±250 dps	±4 g
带宽	10-200 Hz	10-200 Hz
灵敏度因子	0.015 dps/LSB	0.12 mg/LSB
零偏不稳定性	0.0028 dps	±10 mg
偏差温度系数	0.005 dps	0.5 mg
初始偏移误差	±0.5 dps	0.06 mg/℃

2.1.4 机械模块

机器人的三维模型如图 2.5 所示，移动机器人的整体结构包括车体模块、驱动模块、负载板等部分。移动机器人使用箱体式设计，便于组件的安装。车体由底盘和车架组成，底盘的设计取决于机器人内部模块的布局；车架的前侧安装有激光雷达，后侧有充电接口和数据接口；机器人的两侧有急停按钮和电源开关；机器人的前后两侧有防撞条。负载板使用下方 4 个与底盘连接的支撑柱支撑，负载板上有固定的螺纹孔，便于对机器人进行扩展。



图 2.5 移动机器人三维模型结构图

2.1.5 驱动模块

移动机器人采用两轮差速驱动，驱动轮上安装有独立的伺服电机，驱动轮前后有起支撑作用的万向轮，移动机器人采用上海同毅自动化的驱动轮，驱动轮的型号为 TYD160-60SM-04030BA-20，减速机构已经集成到驱动轮中，使得整个驱动轮结构小巧，驱动轮如图 2.6 所示，该产品主要参数如表 2.3 所示。

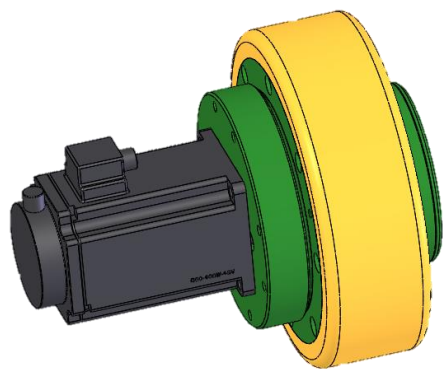


图 2.6 驱动轮

表 2.3 驱动轮的主要参数

	参数项	参数值	单位
驱动电机	额定功率	400	W
	额定电压	48	V(DC)
	额定电流	11	A
	额定转速	3000	r/min
	编码器型号	增量 2500 线	/
驱动轮	额定扭矩	1.27	N·m
	峰值扭矩	2.54	N·m
	减速比	20	/
	直径	162.5	mm
	最大速度	75.36	m/min
	最大扭矩	43	N·m
	最大载荷	350	kg

2.2 移动机器人的软件系统

2.2.1 ROS 简介

ROS(机器人操作系统)具有分布式设计、支持多种开发语言、免费且开源等特点，因此被广泛应用于机器人的研究领域。利用 ROS 搭建机器人的软件系统，不仅能够实现各种功能的模块化设计，而且能够保证软件系统良好扩展性。基于上述条件，本文使用 ROS 进行上位机软件系统的设计。

ROS 中节点是执行任务的进程，ROS 系统一般由多个节点组成，ROS 使用主节点对节点进行管理。Topic(话题)是 ROS 中的异步通信方式，如图 2.7 所示，可以支持一对多通信，即同一个话题可以被多个节点订阅，这种通信方式适用于传感器等需要周期发布的数据，上位机软件系统中使用话题进行通信。

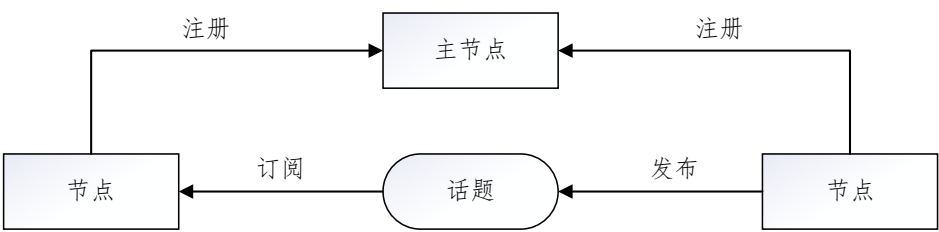


图 2.7 话题通信方式示意图

移动机器人有多个传感器，传感器被安装在移动机器人的不同位置，每个传感器均以自身的坐标系记录测量数据，需要将传感器数据从自身坐标系转换到移动机器人的本体坐标系上才能进行定位和建图。

ROS 采用 TF 树描述传感器坐标系之间的变换关系，TF 树每一个节点表示一个坐标系，每个坐标系仅有一个父坐标系，且存在对应的位姿转换关系。本文自主搭建移动机器人平台的 TF 树如图 2.8 所示，其中 base_footprint 节点代表移动机器人本体的坐标系，laser 节点代表激光雷达的坐标系，base_link 节点代表 IMU 的坐标系。

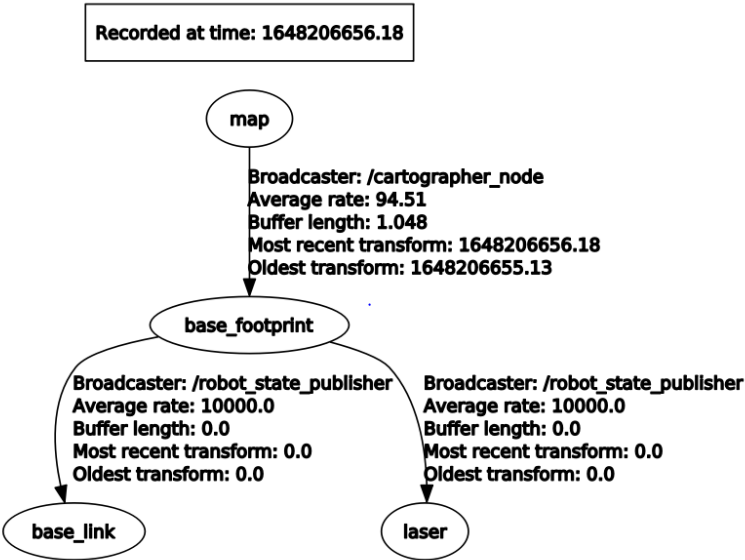


图 2.8 TF 树示意图

2.2.2 软件系统整体架构

移动机器人平台的软件系统由上位机软件系统和下位机软件系统组成，移动机器人平台软件系统整体架构如图 2.9 所示。上位机软件系统的功能包括通过串口与下位机进行通信；接收下位机发送的传感器数据，并以话题的形式进行发布；根据接收到的命令，计算左右轮的转速，然后将左右轮转速指令发送到下位机；进行定位与建图。下位机软

件系统功能包括 IMU 数据采集，编码器数据采集、电量数据采集、运动控制、语音提示等。

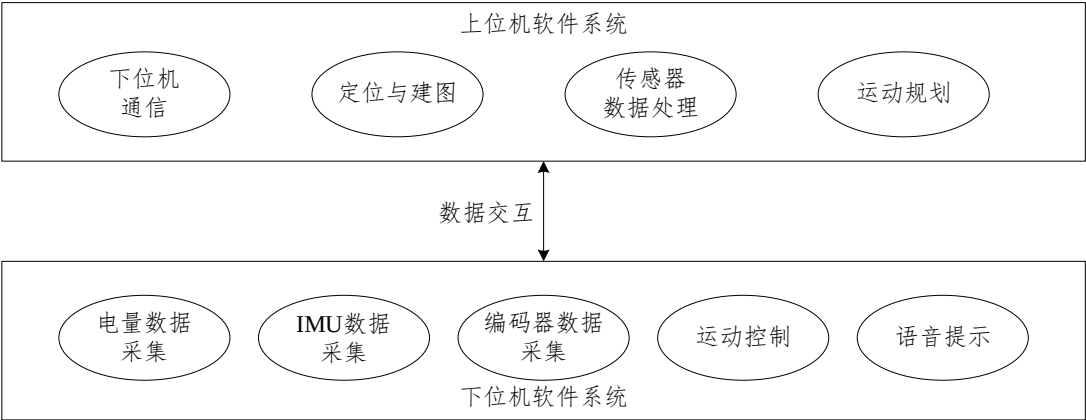


图 2.9 软件系统整体架构

2.2.3 上位机软件系统

上位机软件系统主要功能有：接收传感器数据，并以话题的形式发布；根据传感器的数据，进行同步定位与建图；进行移动机器人的运动规划。上位机软件系统的整体架构如图 2.10 所示，其中 IMU 节点接收下位机传输的 IMU 数据，并发布对应的话题；里程计节点接收下位机传输的编码器数据，完成航迹推算后，发布轮式里程计对应的话题；激光雷达节点接收激光雷达数据，并发布对应的话题；TF 节点定期发布坐标系的位姿变换；运动控制节点对输入的命令进行解析，然后将左右轮转速发送到下位机；SLAM 节点订阅传感器的数据，使用 SLAM 算法进行同步定位和建图，然后以话题的形式发布机器人位置信息和地图信息。

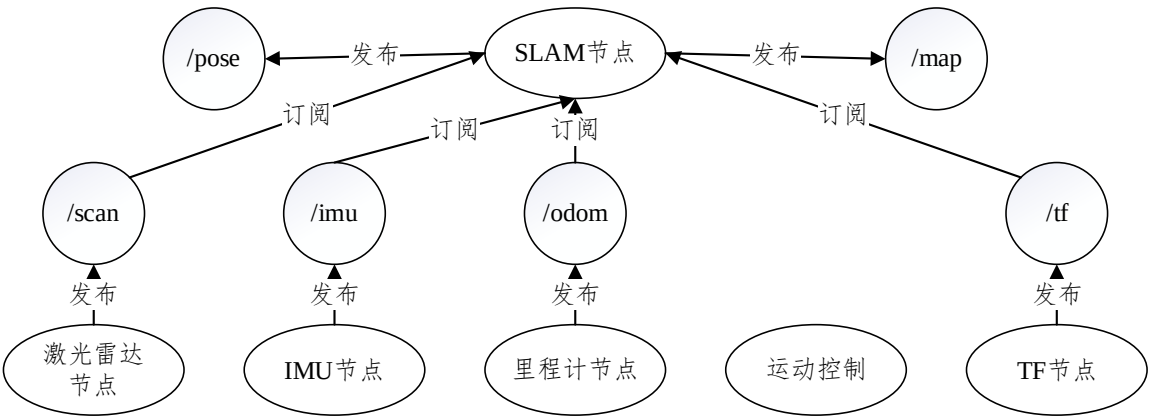


图 2.10 上位机软件系统整体架构

图 2.11 所示为传感器采集模块的工作流程，首先注册话题发布者，然后监听相关报文，根据通信协议对报文进行解析处理，将解析处理结果写入对应的消息格式，最后发

布对应的话题。如图 2.12 所示为运动控制流程，首先在终端监听运动命令，然后在监听到命令后，根据差速模型计算移动机器人左右轮的转速，最后将左右轮的转速通过串口发送到下位机。

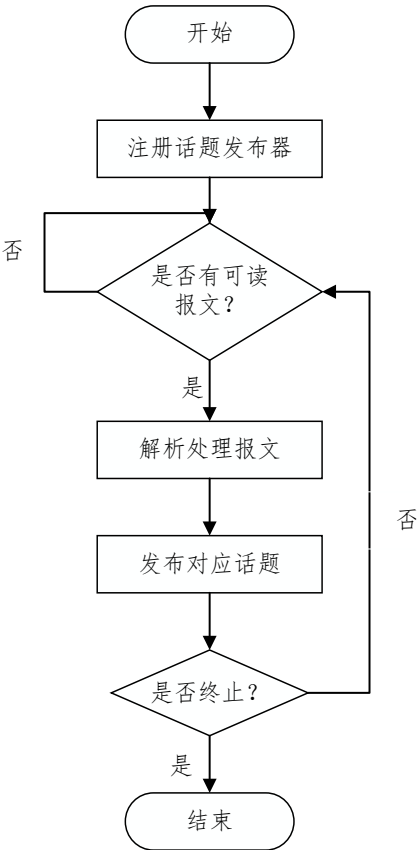


图 2.11 传感器采集模块流程

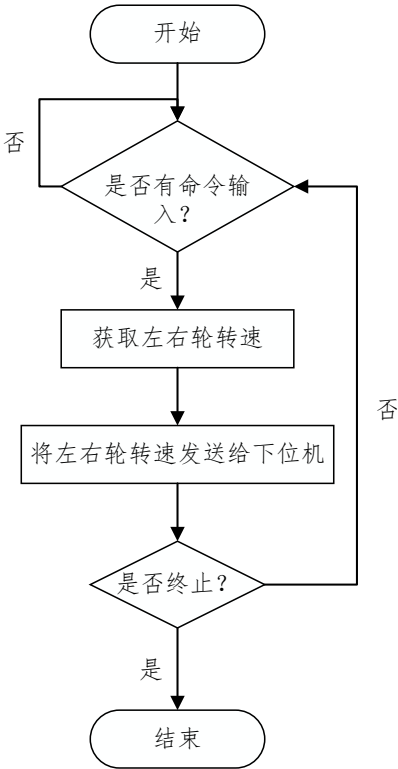


图 2.12 运动控制功能流程

自主搭建的激光 SLAM 系统框架如图 2.13 所示，主要包括前端激光里程计和后端优化两部分。激光里程计以传感器的数据为输入，输出为移动机器人的实时位姿和子图；因为激光里程计的误差会随着时间不断增加，所以减少累计误差对于 SLAM 算法非常重要，因此引入后端优化减少激光里程计的累计误差。本文后续章节将对前端激光里程计算法和后端优化算法进行研究。

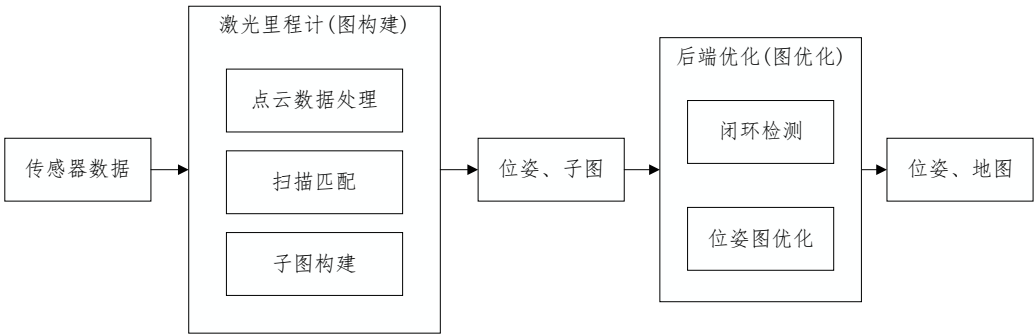


图 2.13 激光 SLAM 系统框架

2.2.4 下位机软件系统

下位机软件系统的主要功能包括以下几个方面：接收上位机的速度数据，使用 CAN 总线发送指令给电机驱动器；采集编码器的数据，将数据发送给上位机；采集 IMU 的数据，将数据发送到上位机；采集电池电量的数据，将数据发送给上位机；接收上位机的命令，控制扬声器发出对应的语音提示。

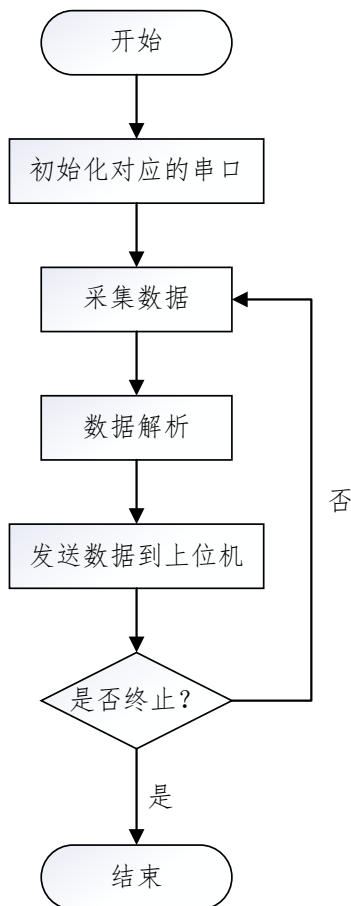


图 2.14 传感器数据采集功能流程

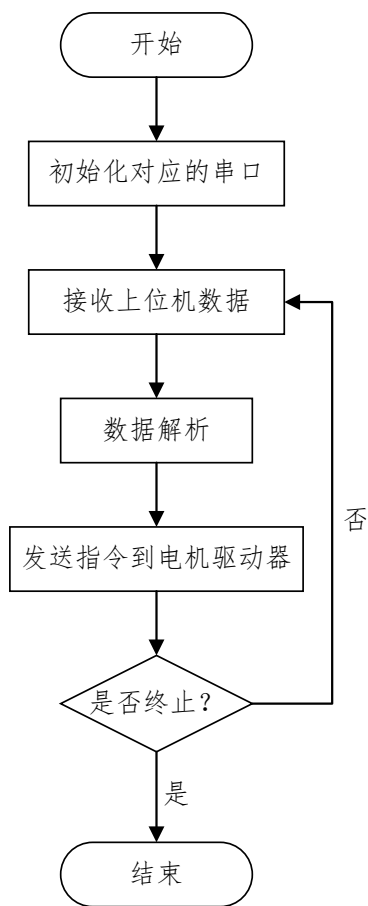


图 2.15 电机控制功能流程

图 2.14 为下位机软件系统中传感器数据采集功能的流程图，首先初始化通讯串口，然后在监听到传感器发送的报文后，对报文进行解析，最后将解析的数据发送到上位机。图 2.15 为下位机软件系统中电机控制功能的流程图，首先初始化通信接口，然后监听串口，在接收到上位机发送的左右轮转速的报文后，将报文转为运动指令，通过 CAN 总线将运动指令发送给电机驱动器。

2.3 本章小结

本章设计了移动机器人平台的硬件系统和软件系统。首先，提出了硬件系统的整体设计方案，对其中的控制模块、感知模块、机械模块和驱动模块进行设计，对关键元器件行选型。然后，设计了软件系统的整体框架，基于 ROS 搭建了上位机软件系统，给出了该系统的整体架构，对其中的定位与建图、传感器数据处理、运动规划等模块进行了设计；最后，给出了下位机嵌入式软件系统的整体功能，对传感器数据采集、运动控制等模块进行了设计。

3 激光里程计算法研究

激光里程计算法作为激光 SLAM 系统的前端，对整个系统的准确性和鲁棒性有重要作用。本章研究了针对差速驱动机器人的激光里程计算法，该算法使用 IMU 数据对激光点云进行畸变矫正，采用差速驱动机器人的位姿估计算法对机器人位姿进行估计，并为帧与图的扫描匹配算法提供初值；采用启发式方法进行关键帧的选取，根据关键帧进行栅格地图构建。

3.1 算法框架

图 3.1 为激光里程计算法框架，算法以激光点云、IMU 数据和里程计数据为输入，输出移动机器人的实时位姿和子图。该算法包括激光点云预处理、扫描匹配、地图构建三个模块，首先，激光点云预处理模块采用 IMU 数据对激光点云进行畸变矫正；然后，扫描匹配模块采用位姿估计算法对机器人位姿进行估计，并为帧与图的扫描匹配算法提供初值；最后，地图构建模块采用启发式方法进行关键帧选取，根据选取的关键帧构建子图。

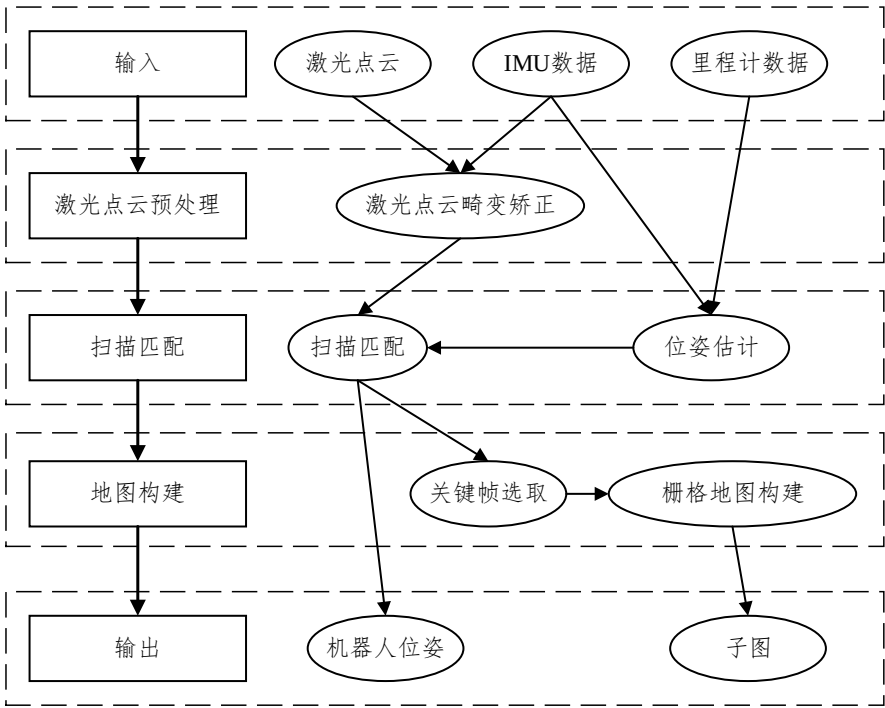


图 3.1 激光里程计算法框架

3.2 激光点云畸变矫正算法研究

一帧激光点云数据中激光点的获取存在时间差, 如果激光雷达扫描的过程中移动机器人位置发生变化, 采集第一个激光点和最后一个激光点时激光雷达所处的位置也会存在相应的位置变化。为了保证激光里程计算法的建图效果和定位精度, 需要对激光点云进行畸变矫正。

对激光点云进行畸变矫正, 需要获取移动机器人在扫描过程中的位姿变化, 因为 IMU 的频率可以达到 100Hz-200Hz, 可以准确反映机器人的位姿变化, 所以利用 IMU 数据对激光点云进行畸变矫正。

使用 IMU 数据对激光点云进行畸变矫正, 将移动机器人的状态定义为^[63]:

$$\mathbf{X}_i = [\mathbf{p}_i \ \mathbf{v}_i \ \mathbf{q}_i \ \mathbf{b}_a^T \ \mathbf{b}_\omega^T]^T, \quad (3.1)$$

其中 $\mathbf{X}_i$ 表示机器人的状态, $\mathbf{p}_i$ 为机器人在世界坐标系下的位置, $\mathbf{v}_i$ 为机器人在世界坐标系下的速度, $\mathbf{q}_i$ 为机器人在世界坐标系下的姿态, $\mathbf{b}_a$ 为 IMU 的加速度漂移, $\mathbf{b}_\omega$ 为 IMU 的角速度漂移。

移动机器人第 j 帧的状态量由 IMU 数据和第 i 帧的状态量离散得到, 计算公式为^[63]:

$$\mathbf{p}_j = \mathbf{p}_i + \sum_{k=i}^j \left[\mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{g}^w \Delta t^2 + \frac{1}{2} \mathbf{q}_k (\hat{\mathbf{a}}_k - \mathbf{b}_a) \Delta t^2 \right], \quad (3.2)$$

$$\mathbf{v}_j = \mathbf{v}_i + \mathbf{g}^w \Delta t_{ij} + \sum_{k=i}^j \mathbf{q}_k (\hat{\mathbf{a}}_k - \mathbf{b}_a) \Delta t, \quad (3.3)$$

$$\mathbf{q}_j = \mathbf{q}_i \prod_{k=i}^j \text{Exp}((\hat{\boldsymbol{\omega}}_k - \mathbf{b}_\omega) \Delta t), \quad (3.4)$$

其中 $\Delta t_{ij} = \sum_{k=i}^j \Delta t$, $\mathbf{g}^w$ 为当前状态的下重力矢量, $\hat{\mathbf{a}}_k$ 为机器人的线加速度矢量, $\hat{\boldsymbol{\omega}}_k$ 为机器人的角速度矢量。为了减少积分次数, 利用 IMU 预积分理论计算机器人的状态变化, 计算公式为^[63]:

$$\Delta \mathbf{p}_{ij} = \mathbf{q}_i^T \left(\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g}^w \Delta t^2 \right) = \sum_{k=i}^j \left[\Delta \mathbf{v}_{ik} \Delta t + \frac{1}{2} \Delta \mathbf{q}_{ik} (\hat{\mathbf{a}}_k - \mathbf{b}_a) \Delta t^2 \right], \quad (3.5)$$

$$\Delta \mathbf{v}_{ij} = \mathbf{q}_i^T (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g}^w \Delta t_{ij}) = \sum_{k=i}^j \Delta \mathbf{q}_{ik} (\hat{\mathbf{a}}_k - \mathbf{b}_a) \Delta t, \quad (3.6)$$

$$\Delta \mathbf{q}_{ij} = \mathbf{q}_i^T \mathbf{q}_j = \prod_{k=i}^j \text{Exp}((\hat{\boldsymbol{\omega}}_k - \mathbf{b}_\omega) \Delta t), \quad (3.7)$$

激光雷达的频率远低于 IMU 的频率，所以两帧激光雷达间会存在多个 IMU 数据，如图 3.2 所示。假设两帧激光雷达数据对应的时间戳为 t_k 和 t_{k+1} ，本文使用预积分计算得到机器人在 t_k 到 t_{k+1} 时间内的相对位姿变换 $\mathbf{T}_{t_k}^{t_{k+1}}$ 。假设在机器人在 $t \in [t_k, t_{k+1}]$ 时间内做匀速运动，基于此就可以使用线性插值求得任意激光点相对于第一个激光点得位姿变换，计算公式为：

$$\mathbf{T}_i = \frac{t_{k+1} - t_i}{t_{k+1} - t_k} \mathbf{T}_{t_k}^{t_{k+1}}, \quad (3.8)$$

其中 t_i 为激光点云中第 i 个激光点的时刻， $\mathbf{T}_i$ 为第 i 激光点相对于第一个激光点的位姿变换矩阵，然后将当前激光点转换到第一个激光点对应的坐标系下，计算公式为：

$$\mathbf{h}_i' = \mathbf{T}_i \mathbf{h}_i, \quad (3.9)$$

其中 $\mathbf{h}_i$ 为畸变矫正前激光点的位置， $\mathbf{h}_i'$ 为畸变矫正后激光点的位置，对激光点云中每一个激光点都进行位姿变换，就完成了一帧激光点云的激光矫正。

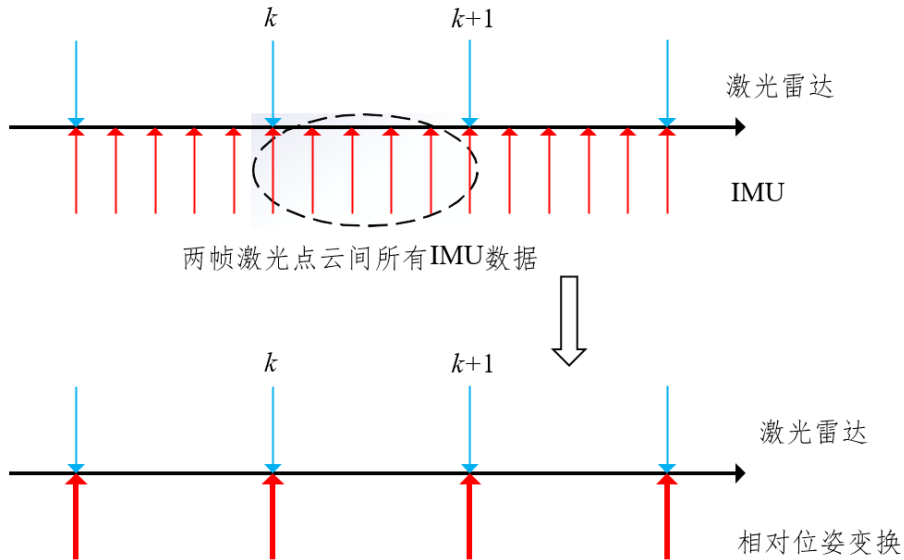


图 3.2 IMU 和激光雷达时间戳示意图

激光点云畸变矫正算法流程如图 3.3 所示，首先订阅 IMU 和激光雷达的数据，针对当前激光点获取对应时间戳下 IMU 数据，根据 IMU 数据进行预积分，然后求解相对

于第一个激光点的位姿变换，将激光点变换到第一个激光点对应的坐标系，对所有激光点重复上述操作，最后输出完成畸变矫正的激光点云数据。

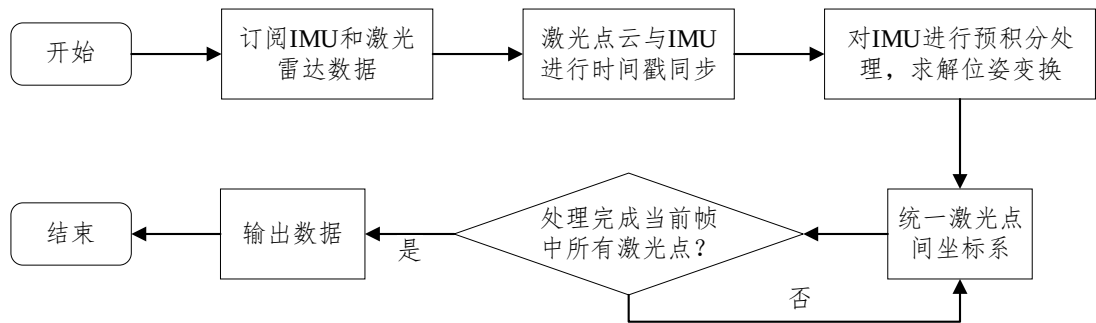


图 3.3 激光点云畸变矫正算法流程图

利用畸变矫正算法对激光点云数据进行畸变矫正，截取机器人旋转时的一帧激光点云进行展示，图 3.4 为激光点云畸变矫正前后对比，红色点云表示未进行畸变矫正的激光点云数据，黄色点云表示畸变矫正后重新发布的激光点云数据，可以看出，畸变矫正后的激光点云数据更接近墙壁真实轮廓。

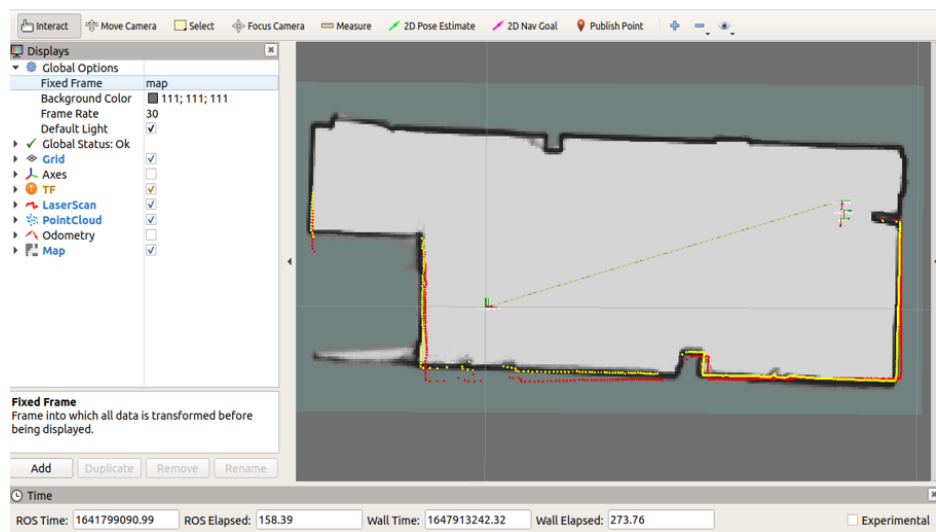


图 3.4 激光点云畸变矫正前后对比图

3.3 激光点云的扫描匹配算法研究

3.3.1 帧与图的扫描匹配算法

扫描匹配算法根据激光点云数据对机器人位姿进行估计，常用的扫描匹配算法有帧与帧的扫描匹配算法、帧与图的扫描匹配算法，帧与帧的扫描匹配算法通过计算相对位

姿变换,估计机器人的位姿,会产生误差;帧与图的扫描匹配算法可以减小误差,因此激光里程计算法采用帧与图的扫描匹配算法。

经过畸变矫正的激光点云仍在激光雷达坐标系,因此在扫描匹配前需要将激光点云从激光雷达坐标系转化到栅格地图坐标系。将激光点云中激光点转换到栅格地图坐标系的公式如下:

$$\mathbf{T}_\xi \mathbf{h} = \begin{pmatrix} \cos \theta_\xi & -\sin \theta_\xi \\ \sin \theta_\xi & \cos \theta_\xi \end{pmatrix} \mathbf{h} + \begin{pmatrix} x_\xi \\ y_\xi \end{pmatrix}. \quad (3.10)$$

其中 $\mathbf{T}_\xi$ 表示为激光雷达坐标系到栅格地图坐标系的转换矩阵; $\mathbf{h}$ 为激光雷达坐标系中激光点的位置; θ_ξ 为激光点云坐标系和栅格地图坐标系之间的夹角; x_ξ 是两个坐标系在 x 方向的平移量; y_ξ 是两个坐标系在 y 方向的平移量。

为了提高帧与图的扫描匹配算法定位精度,需要对离散的栅格地图进行连续化处理,本文采用双三次插值算法对栅格地图进行连续化处理。与双线性插值算法相比,双三次插值算法虽然计算速度稍慢,但是栅格地图连续化处理后更加平滑,可以使得扫描匹配算法的精度更高。如图 3.5 所示,双三次插值算法使用目标点 $\mathbf{h}_m$ 周围最近的 16 个点的状态值来计算 $\mathbf{h}_m$ 点的状态值。

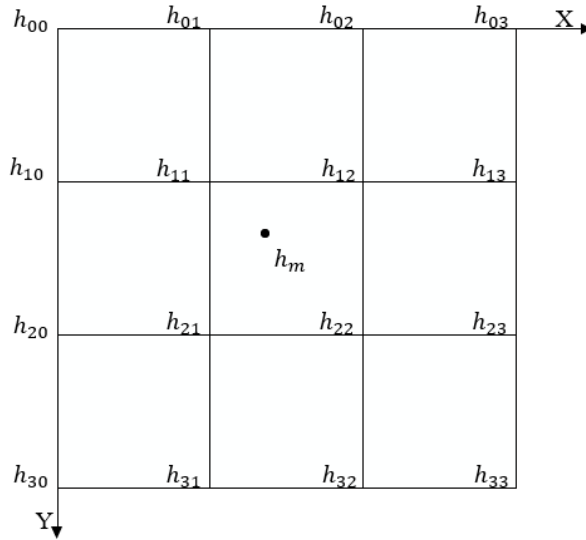


图 3.5 双三次插值示意图

使用双三次插值算法对栅格地图进行连续化处理的过程为:首先构造 Bicubic 函数为:

$$W(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1, & |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a, & 1 < |x| < 2 \\ 0, & |x| \geq 2 \end{cases} \quad (3.11)$$

其中 a 取经验值 -0.5 ，然后使用 Bicubic 函数计算周围 16 个点相对于 $\mathbf{h}_m$ 点的权重值，计算公式为：

$$k_{ij} = W(x_m - x_i)W(y_m - y_j), \quad (3.12)$$

其中 x_i 、 y_j 为周围 16 个点的坐标值， x_m 、 y_m 为对 $\mathbf{h}_m$ 的坐标值向下取整，最后求解周围 16 个点的加权和，计算公式为：

$$M(\mathbf{h}_m) = \sum_{i=0}^3 \sum_{j=0}^3 k_{ij}M(\mathbf{h}_{ij}), \quad (3.13)$$

其中 $M(\mathbf{h}_{ij})$ 表示周围 16 个点的状态值。

在完成栅格地图的连续化后，帧与图的扫描匹配算法需要寻找一个刚性变换 $\mathbf{T}$ ，使得激光点云在连续化处理后的栅格地图上得分最高，由于 $M(\mathbf{h}_m)$ 的取值范围为 $(0,1)$ ，且计算最大值的过程比较复杂，所以将该问题转化为非线性最小二乘优化问题，所以该问题的数学表达为^[27]：

$$\mathbf{T}^* = \arg \min_{\mathbf{T}} \sum_{k=1}^n [1 - M(\mathbf{T}\mathbf{h}_k)]^2. \quad (3.14)$$

其中 $\mathbf{T}$ 表示刚性变换， $\mathbf{h}_k$ 表示激光点在栅格地图下的坐标。

非线性最小二乘优化问题的常用求解方法有：梯度下降法、高斯-牛顿法、列文伯格-马夸尔特法。梯度下降法容易陷入局部最小值，导致整体的收敛速度较慢；高斯-牛顿法在初始点距离极小值点过远时，不保证完全收敛，且计算效率较低；因为列文伯格-马夸尔特法的收敛速度较快，保证收敛，所以在激光里程计中使用列文伯格-马夸尔特法对该问题进行求解。

基于优化方法的扫描匹配算法本质是局部优化问题，所以良好的初值是必须的。初值一般使用位姿估计算法获取，目前位姿估计算法使用位姿变换获取的线速度和角速度对机器人位姿进行位姿估计，或者使用 IMU 获取的线加速度和角速度对机器人位姿进行位姿估计。这些位姿估计算法的准确性较低，不能为优化方法提供良好的初值，针对这种情况提出了差速驱动机器人的位姿估计算法。

3.3.2 差速驱动机器人位姿估计算法

差速驱动机器人动力源于两个带有独立执行机构的驱动轮，执行机构多采用伺服电机，以图 3.6 为例分析差速驱动机器人的运动模型。在图 3.6 中， W_1 和 W_2 为左右两个驱动轮， v_1 和 v_2 为左右两轮的线速度，机器人整体的线速度为 v_c ， r_c 为旋转半径。机器人瞬时线速度 v_c 的计算公式为

$$v_c = \frac{v_1 + v_2}{2}, \quad (3.15)$$

机器人瞬时角速度 ω_c 的计算公式为

$$\omega_c = \frac{v_c}{r_c}. \quad (3.16)$$

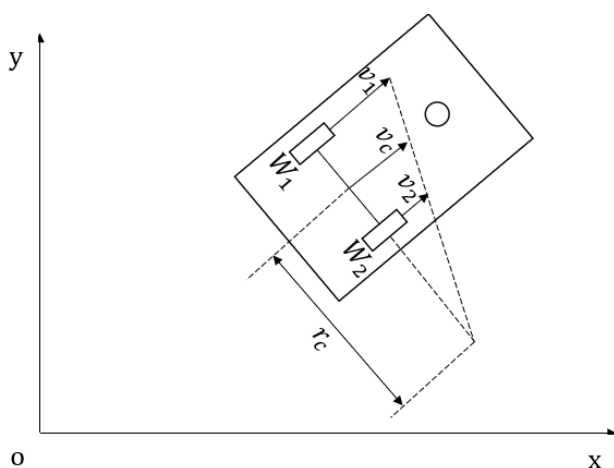


图 3.6 差速驱动机器人的运动模型

差速驱动机器人根据左右轮的线速度差可以分为三种运动状态，分别是直线运行，曲线运动和旋转运动。图 3.7 为差速驱动机器人的三种运动状态。

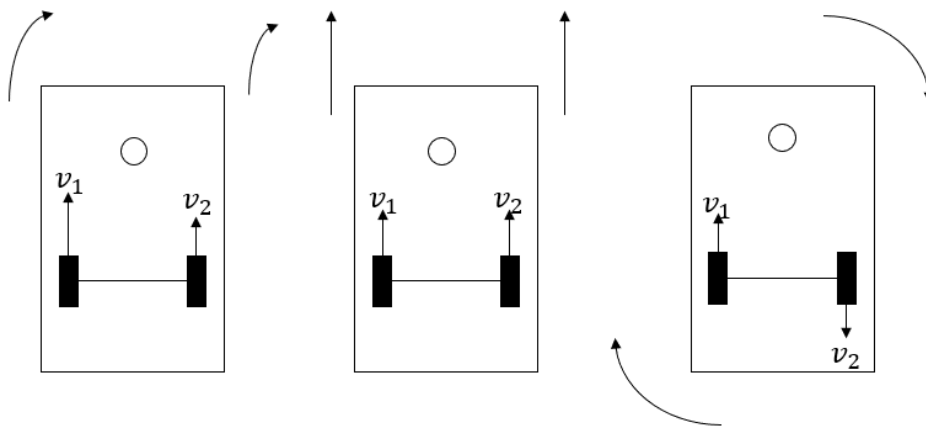


图 3.7 差速驱动机器人的三种运动状态

当 $v_1 > v_2$ 或 $v_1 < v_2$ ，且 $|v_1| \neq |v_2|$ ，机器人做曲线运动；

当 $v_1 = v_2$ 时，机器人做直线运动；

当 $v_1 = -v_2$ 时，机器人以围绕两轮的中心做旋转运动。

在差速驱动机器人实际运动过程，当机器人做直线运动时，机器人的位置发生变化，由于装配误差、地面不平等原因，机器人姿态会发生变化；当机器人做旋转运动时，机器人的姿态发生变换，因为机器人中心和旋转中心不重合，机器人位置会发生变化；当机器人做曲线运动时，机器人的位置和姿态同时发生变化；因此位姿估计算法需要同时估计机器人位置和姿态，图 3.8 为位姿估计算法流程，该算法在进行位姿估计时，需要对机器人的位置和姿态进行估计，下面提出了对机器人位置和姿态进行估计的方法。

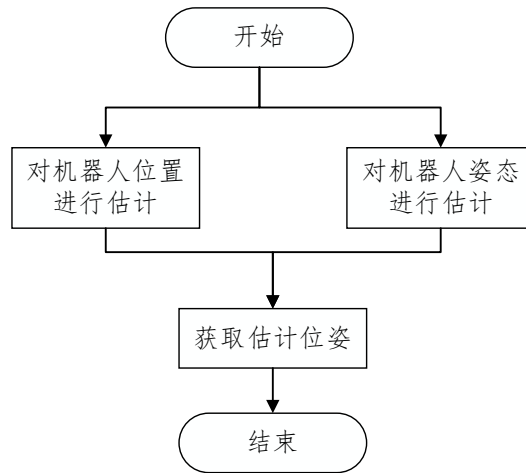


图 3.8 位姿估计算法流程

使用 IMU 线加速度估计移动机器人的位置变化需要进行两次积分，由于 IMU 传感器测量的线加速度存在不可消除的误差，两次积分会放大线加速度存在的误差，因此位姿估计算法中使用里程计的线速度计算移动机器人位置变化。在使用里程计线速度计算移动机器人的位置变化时，认为移动机器人的运动为匀速运动，移动机器人当前时刻位置的计算公式如下：

$$\mathbf{x}_{pos,t} = \mathbf{x}_{pos,t-1} + \mathbf{v}_{odom}\Delta t. \quad (3.17)$$

其中 $\mathbf{x}_{pos,t}$ 为机器人当前时刻的位置， $\mathbf{x}_{pos,t-1}$ 为机器人上一时刻的位置， $\mathbf{v}_{odom}$ 为根据里程计数据获取机器人的速度， Δt 为上一时刻到当前时刻的时间间隔。

里程计获取的角速度是根据编码器数据计算得到的，这就导致通过里程计获取的角速度与真实角速度存在偏差，所以位姿估计算法中使用 IMU 角速度计算移动机器人的姿态变化。在使用 IMU 角速度计算移动机器人的姿态变化时，认为移动机器人的运动为匀速转动，移动机器人当前时刻姿态的计算公式如下：

$$\mathbf{x}_{ori,t} = \mathbf{x}_{ori,t-1} + \omega * \Delta t. \quad (3.18)$$

其中 $\mathbf{x}_{ori,t}$ 为机器人当前时刻的姿态， $\mathbf{x}_{ori,t-1}$ 为机器人上一时刻的姿态， ω 为根据 IMU 数据获取机器人的角速度， Δt 为上一时刻到当前时刻的时间间隔。

3.4 子图更新

3.4.1 关键帧选取

激光里程计算法使用多个子图拼接完成地图，每个子图由一系列连续的激光点云组成。如果每帧激光点云都插入到子图中，会极大增加子图的数量，所以激光里程计只选择关键帧的数据插入到子图中，而对于非关键帧的数据直接舍弃。例如当移动机器人处于静止状态或低速运动时，因为激光点云之间不存在明显差别，此时并不需要将所有的激光点云插入到子图中，只需要选择位姿变化较大的激光点云插入到子图中。因此使用了关键帧，来减少插入到子图中激光点云的数量。

激光里程计使用了简单有效的启发式方法进行关键帧选取。当机器人的位姿变化超过指定阈值时，选择当前激光点云作为关键帧。关键帧之间所有的激光点云都会被丢弃，通过这种方式选择添加到子图的关键帧，可以使用少量的关键帧构建比较清晰地图。激光里程计中平移和旋转阈值分别设置为 0.2 m 和 1° 。

图 3.9 为激光里程计效果图，图中蓝色曲线为机器人的运动轨迹，曲线上的点为子图完成位置，从选取结果来看，子图均匀分布在运动轨迹上，在机器人姿态变化比较大的地方，子图之间的距离近，在机器人姿态变化小的地方，子图之间的距离比较远。子图是由固定数量的关键帧组成，因为子图均匀的分布在运动轨迹上，所以可以认为关键帧也是均匀分布在运动轨迹上。

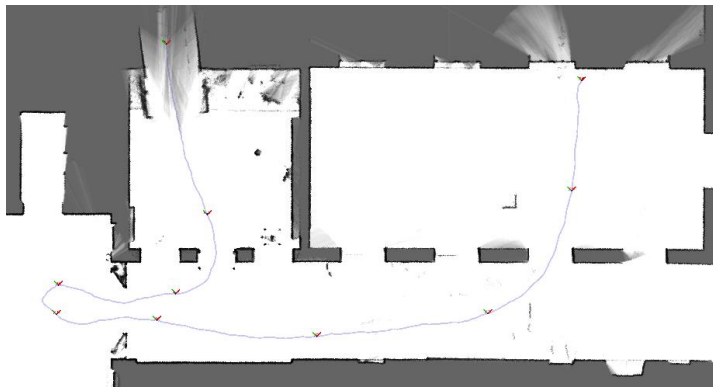


图 3.9 激光里程计效果图

3.4.2 栅格地图构建

常用的地图格式有特征地图、拓扑地图、栅格地图，特征地图可以准确的记录环境的特征，该地图只能用于结构化的场景中，无法用于复杂的场景中；拓扑地图由点和线组成，不能直观反映环境特征；栅格地图已经基本成熟，占用内存适中，可以应用多种场景，因此本文采用栅格地图。

图 3.10 为栅格地图，栅格地图由多个栅格组成，每个栅格的状态相互独立，每个栅格与二值占用变量对应，该变量表示栅格是否被占用， $p(s=1)$ 表示栅格被占用的概率，对应地图中障碍物； $p(s=0)$ 表示栅格空闲的概率，对应地图中可通行的区域；其中

$$p(s=1) + p(s=0) = 1. \quad (3.19)$$

栅格状态取决于被占用概率和空闲概率的比值^[64]

$$odds(s) = \frac{p(s=1)}{1 - p(s=1)}, \quad (3.20)$$

其中 $odds(s)$ 为计算栅格状态的函数。

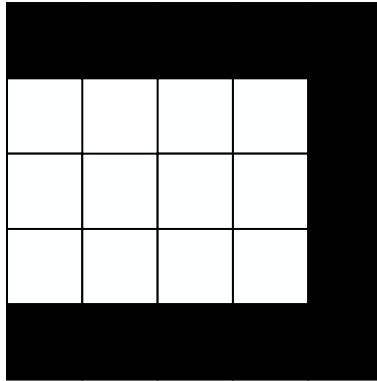


图 3.10 栅格地图

图 3.11 为激光点云观测模型，激光点对栅格有两种观测状态，激光点落在栅格上时，对栅格的观测值为 1；激光点和激光雷达原点之间的连线穿过栅格时，对栅格的观测值为 0，可以引入观测值 $z \sim \{0, 1\}$ 。在激光点云数据插入栅格前，栅格的状态为 $odds(s)$ ，根据观测值更新栅格状态，采用条件概率公式可得

$$odds(s|z) = \frac{p(z|s=1)}{p(z|s=0)} odds(s). \quad (3.21)$$

其中 $odds(s|z)$ 为更新后栅格的状态, $\frac{p(z|s=1)}{p(z|s=0)}$ 为观测模型, 根据观测值不同, 观测模型存在不同的取值, 当 $z=1$ 时, 观测模型为 $\frac{p(z=1|s=1)}{p(z=1|s=0)}$, 当 $z=0$ 时, 观测模型为 $\frac{p(z=0|s=1)}{p(z=0|s=0)}$ 。

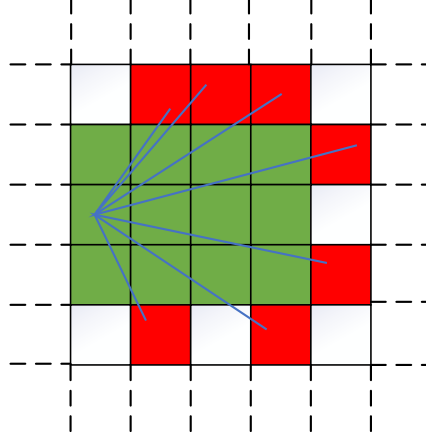


图 3.11 激光点云观测模型

经过上述建模后, 栅格的更新可以简化为

$$odds(s)_{new} = \frac{p(z|s=1)}{p(z|s=0)} odds(s)_{old}, \quad (3.22)$$

其中 $odds(s)_{old}$ 为更新前的栅格状态, $odds(s)_{new}$ 为更新后的栅格状态。在栅格第一次被观测到时, 栅格的状态更新公式为

$$odds(s)_{new} = \frac{p(z|s=1)}{p(z|s=0)}. \quad (3.23)$$

3.5 仿真实验

3.5.1 评价标准

本文使用绝对轨迹误差对算法的定位精度进行评价, 绝对轨迹误差可以计算估计位姿与真实位姿的差值, 能够直观反映算法精度和轨迹的全局一致性。为了计算绝对轨迹误差, 需要先将真实轨迹和估计轨迹统一到相同坐标系下, 计算两段轨迹在同一时刻下的误差值, 轨迹误差的计算公式为:

$$F_i = Q_i^{-1} P_i. \quad (3.24)$$

其中 $\mathbf{Q}_i$ 为 i 时刻机器人真实位姿, $\mathbf{P}_i$ 为 i 时刻机器人估计位姿。为了对轨迹整体误差进行计算, 使用所有时刻绝对轨迹误差平移分量的均方根误差来衡量整个轨迹误差。均方根误差的计算公式为:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n \|\text{trans}(\mathbf{F}_i)\|^2}. \quad (3.25)$$

其中 $\text{trans}(\mathbf{F}_i)$ 表示绝对轨迹误差的平移分量。

3.5.2 激光里程计实验

为了验证本文激光里程计算法的建图效果和定位精度, 使用公开的数据集进行实验, 并与主流的开源激光里程计算法进行对比。目前主流开源的激光里程计算法有 Hector、LOAM、A-LOAM、LeGO-LOAM 等, 其中 LOAM、A-LOAM 和 LeGO-LOAM 适用于 3D 激光雷达, Hector 适用于 2D 的激光雷达, 因为本文的激光里程计算法适用于 2D 激光雷达, 所以采用本文的激光里程计算法与 Hector 进行比较。

本文选择公开数据集进行实验, 数据集使用 Magazion 机器人在走廊中录制, 数据集有 2D 激光雷达数据、IMU 数据和里程计数据, 其中激光雷达的频率为 15Hz, IMU 的频率为 250Hz, 里程计的频率为 125Hz, 数据集的具体信息如图 3.12 所示。

```
path:          hw1.bag
version:       2.0
duration:      2:17s (137s)
start:         Feb 19 2018 07:56:41.70 (1519055801.70)
end:          Feb 19 2018 07:58:59.44 (1519055939.44)
size:         40.4 MB
messages:     79024
compression:  none [53/53 chunks]
types:
  nav_msgs/Odometry      [cd5e73d190d741a2f92e81eda573aca7]
  sensor_msgs/Imu        [6a62c6daae103f4ff57a132d6f95cec2]
  sensor_msgs/LaserScan  [90c7ef2dc6895d81024acba2ac42f369]
  tf2_msgs/TFMessage     [94810edda583a504dfda3829e70d7eec]
topics:
  /imu/data_raw_transformed 34369 msgs : sensor_msgs/Imu
  /odom                    13771 msgs : nav_msgs/Odometry
  /scan_front              1748 msgs : sensor_msgs/LaserScan
  /scan_rear               1748 msgs : sensor_msgs/LaserScan
  /tf                      27386 msgs : tf2_msgs/TFMessage
  /tf_static                2 msgs : tf2_msgs/TFMessage
```

图 3.12 数据集的详细信息

图 3.13 是本文的激光里程计算法构建的地图, 图 3.14 是 Hector 算法构建的地图。本文激光里程计算法构建地图的边界连续且清晰, 场景特征明显, 而 Hector 算法构建地图的边界不清晰不连续, 场景特征不清晰。表明本文激光里程计算法的建图效果高于 Hector 算法的建图效果。



图 3.13 本文激光里程计算法构建的地图



图 3.14 Hector 算法构建的地图

为了比较两种算法的定位精度，对两种算法的估计轨迹进行分析，不同算法估计轨迹与真实轨迹对比如图 3.15 所示。图中黑色实线为机器人的真实轨迹，红色点划线为本文激光里程计算法的估计轨迹，蓝色虚线为 Hector 算法的估计轨迹，可以发现本文激光里程计算法的估计轨迹比 Hector 算法的估计轨迹更接近真实轨迹。

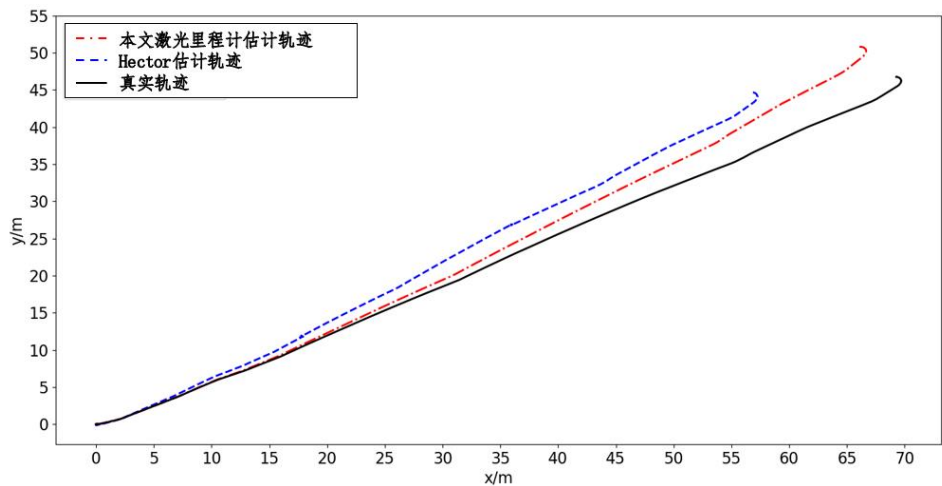


图 3.15 不同算法估计轨迹对比

为了量化不同算法估计轨迹与真实轨迹的误差，采用 `evo` 计算绝对轨迹误差，结果如表 3.1 所示，其中本文激光里程计算法的均方根误差、误差平方、标准差均小于 Hector 算法，表明本文激光里程计算法的估计轨迹，比 Hector 算法的估计轨迹更接近真实轨迹，表明本文激光里程计算法的定位精度优于 Hector 算法。

表 3.1 绝对轨迹误差

算法	均方根误差 (m)	误差平方和 (m ²)	标准差 (m)
Hector	7.877911	108421.422174	5.069659
本文激光里程计	2.424082	74797.792967	1.779899

通过上述的实验分别验证了本文激光里程计算法的建图效果和定位精度均高于 Hector 算法，证明了本文激光里程计算法的有效性。

3.5.3 位姿估计算法实验

为了验证本文提出的位姿估计算法可以为扫描匹配算法提供准确度更高的初值，使用上文的数据集进行实验，与使用 **LEGO-LOAM** 位姿估计方案的激光里程计算法（以下简称对照激光里程计算法）进行对比。对照激光里程计算法除位姿估计算法外，其他部分与本文的激光里程计算法保持一致。

图 3.16 是对照激光里程计算法构建的地图，对照激光里程计算法构建地图的边界连续且清晰，场景特征清晰，与图 3.13 对比发现，对照激光里程计算法和本文激光里程计算法在建图效果上没有明显差别。



图 3.16 对照激光里程计算法构建的地图

为了比较两种算法的定位精度，对两种算法的估计轨迹进行分析，不同算法估计轨迹与真实轨迹对比如图 3.17 所示，图中黑色实线代表机器人真实轨迹，蓝色虚线为本文激光里程计算法的估计轨迹，红色点划线为对照激光里程计算法的估计轨迹，可以发现本文的激光里程计算法的估计轨迹，比对照激光里程计算法的估计轨迹更接近真实轨迹。

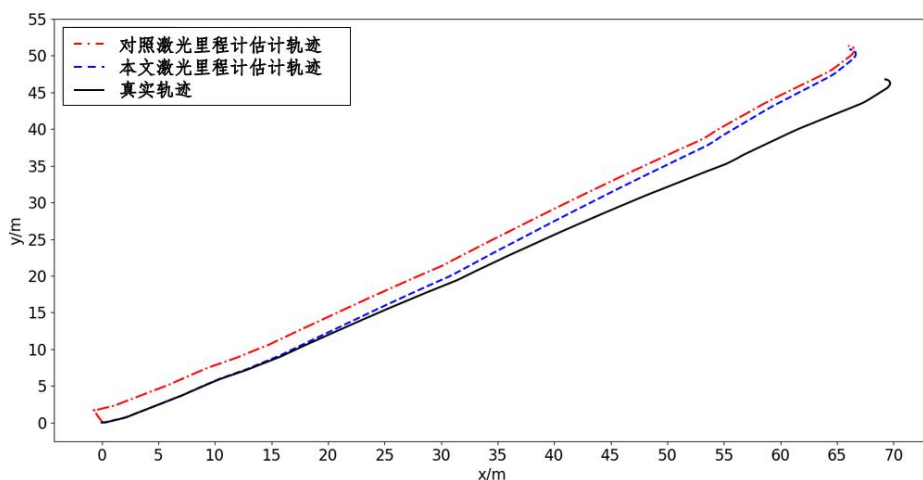


图 3.17 不同算法估计轨迹对比

为了量化不同算法估计轨迹与真实轨迹的误差，采用 **evo** 计算绝对轨迹误差，结果如表 3.2 所示，本文激光里程计算法的均方根误差、误差平方和、标准差均小于对照激

光里程计算法,说明本文激光里程计算法的估计轨迹,比对照激光里程计的估计轨迹更接近真实轨迹,可见本文的激光里程计算法的定位精度高于对照激光里程计算法。

表 3.2 绝对轨迹误差

算法	均方根误差 (m)	误差平方和 (m^2)	标准差 (m)
对照激光里程计	3.260418	135366.528564	1.779899
本文激光里程计	2.424082	74797.792967	1.388342

本文激光里程计算法的定位精度高于对照激光里程计算法,表明本文激光里程计中扫描匹配算法计算的位姿更加准确。扫描匹配算法计算的位姿取决于位姿估计算法提供的初始值和扫描匹配算法,因为二者使用的扫描匹配算法相同,可以说明本文提出位姿估计算法可以为扫描匹配算法提供准确度更高的初值。

3.6 本章小结

本章对激光里程计算法进行研究,搭建了激光里程计系统,相较于传统的激光里程计,本文激光里程计算法具有更好的建图效果和更高定位精度。首先采用 IMU 数据对激光点云进行畸变矫正;然后使用位姿估计算法对机器人位姿进行估计,并为帧与图扫描匹配算法提供初值;最后进行关键帧的选取,根据选取的关键帧进行地图构建。通过激光里程计仿真实验,证明本文激光里程计具有更好的性能;与使用 LEGO-LOAM 位姿估计方案的激光里程计算法进行对比,本文激光里程计估计轨迹与真实轨迹的相对误差下降了 25%,证明本文位姿估计算法可以为扫描匹配算法提供准确度更高的初值。

4 后端优化算法研究

为了减小了激光里程计的累计误差，提高激光 SLAM 系统的建图性能，本章对后端优化算法进行研究。该算法首先采用像素匹配进行闭环检测，使用分支定界加速闭环检测，然后根据检测到的闭环构建位姿图，建立目标函数，最后使用列文伯格-马夸尔特法进行求解，实现了位姿图优化。

4.1 算法框架

激光里程计的误差会随着时间不断增加，后端优化算法的主要目标是减少激光里程计的累计误差，后端优化方法有基于贝叶斯滤波的后端优化方法和基于图优化的后端优化方法，基于贝叶斯滤波的后端优化方法中机器人当前时刻的状态与上一时刻机器人的状态有关，因此当机器人状态出现偏差时，偏差无法消除，会导致构建的地图出现错误，所以该方法无法构建大场景的地图。基于图优化的后端优化算法求解速度快，鲁棒性好、可以构建大场景的地图，因此本文采用基于图优化的后端优化方法。

图 4.1 为激光 SLAM 的后端优化算法框架，该算法以机器人位姿和子图为输入，输出全局地图。该算法由闭环检测和位姿图优化两部分组成，闭环检测部分采用像素匹配进行闭环检测，为了保证 SLAM 算法的实时性，使用分支定界加速闭环检测；位姿图优化部分根据检测到的闭环构建位姿图，建立目标函数，并采用列文伯格-马夸尔特法对目标函数进行求解。

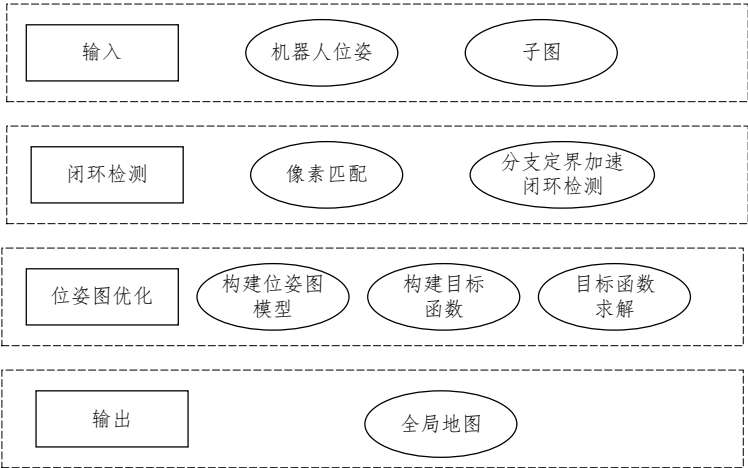


图 4.1 激光 SLAM 的后端优化算法框架

4.2 闭环检测算法

常用的闭环检测算法有帧与帧的闭环检测算法、帧与子图的闭环检测算法、子图与子图的闭环检测算法。帧与帧的闭环检测算法在相似场景中进行闭环检测，可能会检测到错误的闭环，子图与子图的闭环检测算法计算量很大，帧与子图的闭环检测算法的计算量中等，闭环检测的效果也比较好，因此本文采用帧与子图的闭环检测算法。

4.2.1 基于像素匹配的闭环检测

因为激光点云具有旋转不变性，当存在相同的路标时，激光点云和栅格地图的概率分布具有相似性，因此本文基于像素匹配进行闭环检测。基于像素匹配的闭环检测方法，需要寻找激光点云和栅格地图的最优匹配，然后根据最优匹配的结果判断激光点云和栅格地图中是否存在相同路标。如果激光点云和栅格地图中存在相同路标，则存在闭环，否则不存在闭环。

寻找激光点云和栅格地图的最优匹配，需要计算激光点在栅格地图上的得分，本文计算激光点在栅格地图的得分时考虑了栅格的状态，当激光点落在占用的栅格时，激光点的得分为栅格的被占用概率；当激光点落在空闲的栅格时，激光点的得分为 0。当栅格的被占用概率大于等于空闲概率时，栅格的状态为占用；当栅格的被占用概率为小于空闲概率时，栅格的状态为空闲；计算激光点在栅格地图上得分的公式为：

$$S(\mathbf{h}_m) = \begin{cases} p(s=1), & p(s=1) \geq p(s=0) \\ 0, & p(s=1) < p(s=0) \end{cases} \quad (4.1)$$

其中 $\mathbf{h}_m$ 为激光点在栅格地图上的位置。

寻找激光点云和栅格地图的最优匹配，就是在位姿窗口中寻找位姿变换使得激光点云和栅格地图匹配的得分最大，其计算公式为^[27]：

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in W} \sum_{k=1}^n S(T_{\mathbf{x}} \mathbf{h}_k), \quad (4.2)$$

其中 W 是位姿搜索窗口， $\mathbf{x}$ 为搜索窗口中的位姿， $T_{\mathbf{x}}$ 为位姿变换矩阵， $\mathbf{h}_k$ 为激光点的位置，函数 S 计算激光点在栅格地图上的得分。

为了寻找最优像素匹配，需要搜索整个窗口，本文利用线性搜索和角度搜索，其中线性步长设为地图分辨率 r ，假设角度步长 δ_{θ} 。为了保证进行一次角度旋转后，激光点

云中最远的激光点没有移动超过一个栅格的宽度，因此 δ_θ 可以使用余弦定理可推导得到：

$$d_{max} = \max_{k=1,\dots,K} \|\mathbf{h}_k\|, \quad (4.3)$$

$$\delta_\theta = \arccos\left(1 - \frac{r^2}{2d_{max}^2}\right), \quad (4.4)$$

其中 d_{max} 为激光点云中最远的激光点到激光原点最远距离。

为了搜索整个窗口，根据搜索步长计算在当前搜索窗口中能够进行移动的最大步数，计算公式如下：

$$w_x = \left\lfloor \frac{W_x}{r} \right\rfloor, \quad w_y = \left\lfloor \frac{W_y}{r} \right\rfloor, \quad w_\theta = \left\lfloor \frac{W_\theta}{\delta_\theta} \right\rfloor. \quad (4.5)$$

其中 W_x 、 W_y 、 W_θ 代表搜索窗口的大小， w_x 、 w_y 、 w_θ 代表在搜索窗口进行移动的最大步数。位姿搜索窗口是以激光里程计估计位姿 $\mathbf{x}_0$ 为中心的搜索窗口，计算公式如下：

$$\bar{\mathcal{W}} = \{-w_x, \dots, w_x\} \times \{-w_y, \dots, w_y\} \times \{-w_\theta, \dots, w_\theta\}, \quad (4.6)$$

$$\mathcal{W} = \{\mathbf{x}_0 + (rj_x, rj_y, \delta_\theta j_\theta) | (j_x, j_y, j_\theta) \in \bar{\mathcal{W}}\}, \quad (4.7)$$

其中 $\bar{\mathcal{W}}$ 是在搜索窗口中进行移动的集合， j_x 是在 x 方向移动的步数， j_y 是在 y 方向移动的步数， j_θ 是旋转步数。

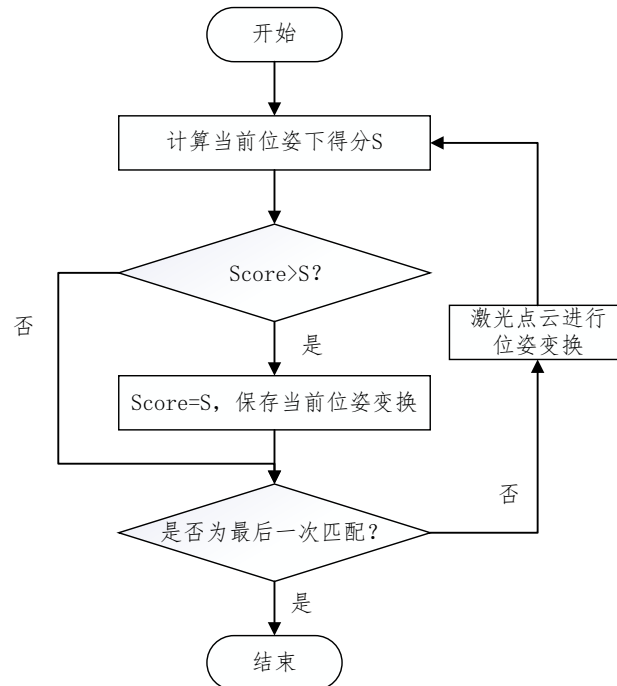


图 4.2 暴力搜索流程

对于上述的搜索问题，最常用的是暴力搜索的方法，暴力搜索流程如图 4.2 所示。首先计算当前位姿变换下激光点云和栅格地图的匹配得分，当得分高于最高得分时，保存当前的得分和位姿变换。当得分小于最高得分，如果是最后一次匹配，流程结束；如果不是最后一次匹配，激光点云进行位姿变换，重新计算得分。

4.2.2 基于分支定界的闭环检测

由于暴力搜索的方法时间复杂度高，效率低下，在大场景环境中，无法保证 SLAM 算法的实时性，因此使用分支定界加速匹配，该方法将可行解的子集看作树的一个节点，子树代表一种可能解的子集，叶子节点代表一种可行解，当节点有上限时，就可以得到最优解。分支定界方法加速匹配包括：节点选择，分支规则，计算上限，剪枝。

1) 节点选择

分支定界算法采用深度优先搜索对树进行搜索，且每个节点只能访问一次。算法的效率很大程度上取决于节点的数量，因此引入了分数阈值，当节点的得分低于该阈值时，不对该节点进行后续搜索。并且对子节点的访问顺序进行设计，首先计算每个子节点的得分，然后对所有的子节点根据得分从大到小排序，最后按照顺序依次访问，对小于父节点得分的子节点不访问。

2) 分支规则

树中的每个节点由一个整数元组表示 $c = (c_x, c_y, c_\theta, c_h)$ ， c_x 是节点表示的可行解在 x 方向移动的步数， c_y 是节点表示的可行解在 y 方向移动的步数， c_θ 是节点表示的可行解的旋转步数， c_h 代表节点高度。高度为 c_h 的节点最多包含 $2^{c_h} \times 2^{c_h}$ 种可行解，此外还有特定的旋转，因此在高度为 c_h 的节点包含的可行解的集合^[27] $\overline{\mathcal{W}}_c$ ：

$$\overline{\mathcal{W}}_c = (\{(j_x, j_y) | c_x \leq j_x < c_x + 2^{c_h}, c_y \leq j_y < c_y + 2^{c_h}\} \times \{c_\theta\}), \quad (4.8)$$

$$\overline{\mathcal{W}}_c = \overline{\mathcal{W}}_c \cap \mathcal{W}, \quad (4.9)$$

其中 $\overline{\mathcal{W}}_c$ 为包含最多可行解的情况。在节点的高度 $c_h=0$ 时，叶子节点对应的可行解 $\mathbf{x}_c = \mathbf{x}_0 + (rc_x, rc_y, \delta_\theta c_\theta)$ 。

根节点的计算公式为^[27]

$$\overline{\mathcal{W}}_{0,x} = \{-w_x + 2^{c_0} j_x, 0 \leq 2^{c_0} j_x \leq 2w_x\}, \quad (4.10)$$

$$\bar{\mathcal{W}}_{0,y} = \{-w_y + 2^{c_0}j_y, 0 \leq 2^{c_0}j_y \leq 2w_y\}, \quad (4.11)$$

$$\bar{\mathcal{W}}_{0,\theta} = \{j_\theta: -w_\theta \leq j_\theta \leq w_\theta\}, \quad (4.12)$$

$$\mathcal{C}_0 = \bar{\mathcal{W}}_{0,x} \times \bar{\mathcal{W}}_{0,y} \times \bar{\mathcal{W}}_{0,\theta} \times \{c_0\}. \quad (4.13)$$

对于高度 $c_h > 1$ 的节点 c ，可以将其分为高度为 $c_h - 1$ 的四个子节点

$$\mathcal{Z}_c = \left((\{c_x, c_x + 2^{c_h-1}\} \times \{c_y, c_y + 2^{c_h-1}\} \times \{c_\theta\}) \cap \bar{\mathcal{W}} \right) \times \{c_h - 1\}, \quad (4.14)$$

其中 $\mathcal{Z}_c$ 为节点 c 的子节点集合。

3) 计算上限

节点 c 代表的可行解子集得分上限计算公式为^[27]:

$$\begin{aligned} score(c) &= \sum_{k=1}^n \max_{x \in \bar{\mathcal{W}}_c} S(T_x \mathbf{h}_k) \\ &\geq \sum_{k=1}^n \max_{x \in \bar{\mathcal{W}}_c} S(T_x \mathbf{h}_k) \\ &\geq \max_{x \in \bar{\mathcal{W}}_c} \sum_{k=1}^n S(T_x \mathbf{h}_k). \end{aligned} \quad (4.15)$$

为了提高计算效率，对栅格地图进行预处理得到 S^{c_h} ， S^{c_h} 和 S 结构相同， S^{c_h} 的计算公式如下^[27]:

$$S^{c_h}(x, y) = \max_{\substack{x' \in [x, x+r(2^h-1)] \\ y' \in [y, y+r(2^h-1)]}} S(x', y'). \quad (4.16)$$

因此当节点的高度为 c_h 时，节点的得分上限计算公式为^[27]:

$$score(c) = \sum_{k=1}^n S^{c_h}(T_x \mathbf{h}_k). \quad (4.17)$$

4) 剪枝

激光点云根据节点代表的可行解进行位姿变换，根据落在空闲栅格上激光点的数量进行剪枝，设置了数量阈值，当落在空闲栅格上激光点的数量大于数量阈值时，不对该节点进行后续搜索，否则进行后续搜索。统计激光点落在空闲栅格上数量的计算公式为:

$$count(c) = \sum_{k=1}^n d(T_x \mathbf{h}_k), \quad (4.18)$$

其中函数 d 判断激光点是否落在空闲栅格上，其计算公式为:

$$d(\mathbf{h}_m) = \begin{cases} 1 & p(s=1) \geq p(s=0) \\ 0 & p(s=1) < p(s=0) \end{cases} \quad (4.19)$$

其中 $\mathbf{h}_m$ 为位姿变换后激光点在栅格地图上的位置。

4.3 位姿图优化算法

位姿图优化算法根据检测到的闭环，建立图优化问题模型，构建目标函数，使用非线性优化方法对目标函数进行求解。如果闭环检测算法检测到节点 $\mathbf{x}_i$ 与 $\mathbf{x}_j$ 间存在闭环，就将闭环约束添加到位姿图中，图 4.3 为闭环约束的示意图。

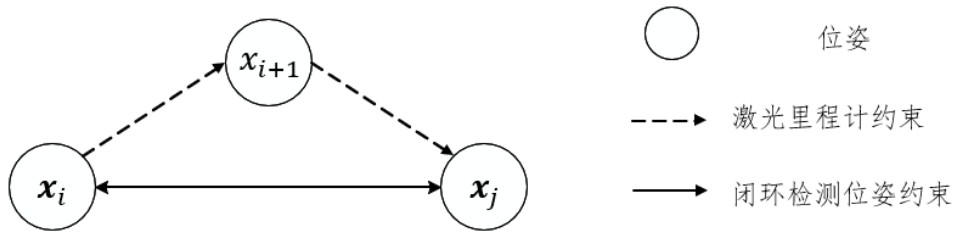


图 4.3 闭环约束示意图

假设节点 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 为激光里程计构建的位姿， $\mathbf{T}_{ij}$ 为根据闭环约束得到的节点 $\mathbf{x}_i$ 和节点 $\mathbf{x}_j$ 之间的位姿变换；此外，节点 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 通过激光里程计的估计位姿进行连接，使用激光里程计的信息可以计算节点 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 的相对位姿变换 $\mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)$ ，计算公式为：

$$\mathbf{h}(\mathbf{x}_i, \mathbf{x}_j) \equiv \begin{cases} \mathbf{R}_i^T (\mathbf{t}_j - \mathbf{t}_i) \\ \theta_j - \theta_i \end{cases} \quad (4.20)$$

其中 $\mathbf{t}_j$ 为机器人的位置； θ_j 为机器人的位姿； $\mathbf{R}_i^T$ 为旋转矩阵，其计算公式为：

$$\mathbf{R}_i^T = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{bmatrix} \quad (4.21)$$

因为激光里程计的误差以及观测噪声的存在，观测值 $\mathbf{T}_{ij}$ 和期望值 $\mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)$ 之间会存在误差， $\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$ 为节点 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 之间的误差项，误差的计算公式为^[65]：

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{T}_{ij} - \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j). \quad (4.22)$$

假设机器人位姿约束总数为 C ，考虑到不同观测信息的信息矩阵，基于图优化模型的目标函数可以表示为^[65]：

$$\mathbf{F}(\mathbf{x}) = \sum_{\langle i, j \rangle \in C} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j), \quad (4.23)$$

其中 $\boldsymbol{\Omega}_{ij}$ 为观测信息协方差矩阵的逆矩阵。后端优化问题的目标就是找到最优位姿配置 $\boldsymbol{x}^*$ ，使得误差函数最小^[65]：

$$\boldsymbol{x}^* = \arg \min_{\boldsymbol{x}} F(\boldsymbol{x}). \quad (4.24)$$

为了采用列文伯格-马夸尔特法对上述优化问题进行求解，使用一阶泰勒公式将误差函数在初值 $\tilde{\boldsymbol{x}}$ 处展开，误差函数展开式为^[65]：

$$\boldsymbol{e}_{ij}(\tilde{\boldsymbol{x}}_i + \Delta \boldsymbol{x}_i, \tilde{\boldsymbol{x}}_j + \Delta \boldsymbol{x}_j) \approx \boldsymbol{e}_{ij} + \boldsymbol{J}_{ij} \Delta \boldsymbol{x}. \quad (4.25)$$

其中 $\boldsymbol{J}_{ij}$ 为 $\boldsymbol{e}_{ij}$ 在 $\tilde{\boldsymbol{x}}$ 处的雅可比矩阵。使用列文伯格-马夸尔特法将非线性优化问题转化为线性问题，线性问题的表达式为^[65]：

$$(\boldsymbol{H} + \lambda \boldsymbol{H}_{diag}) \Delta \boldsymbol{x} = \boldsymbol{g}. \quad (4.26)$$

其中 λ 为系数， $\boldsymbol{H}_{diag}$ 为对角线矩阵，式中 $\boldsymbol{H}$ 与 $\boldsymbol{g}$ 的表达式分别为^[65]：

$$\boldsymbol{H} = \sum_{ij} \boldsymbol{J}_{ij}^T \boldsymbol{\Omega}_{ij} \boldsymbol{J}_{ij}, \quad (4.27)$$

$$\boldsymbol{g} = - \sum_{ij} \boldsymbol{e}_{ij}^T \boldsymbol{\Omega}_{ij} \boldsymbol{J}_{ij}, \quad (4.28)$$

其中 $\boldsymbol{H}$ 与 $\boldsymbol{g}$ 为稀疏矩阵，利用稀疏矩阵的性质可以快速求得 $\Delta \boldsymbol{x}$ ，不断迭代直至收敛得到最优位姿

$$\boldsymbol{x}^* = \tilde{\boldsymbol{x}} + \Delta \boldsymbol{x}^*. \quad (4.29)$$

本文采用列文伯格-马夸尔特法对目标函数求解，具体步骤为：

- 1) 如果 $\lambda < 10^{-4}$ ，设置 λ 为上一次的值；
- 2) 根据 $\tilde{\boldsymbol{x}}$ 构造 $\boldsymbol{H}$ 和 $\boldsymbol{g}$ ；
- 3) 求解 $(\boldsymbol{H} + \lambda \boldsymbol{H}_{diag}) \Delta \boldsymbol{x} = \boldsymbol{g}$ ；
- 4) 使用 $\Delta \boldsymbol{x}$ 更新位姿， $\boldsymbol{x} = \boldsymbol{x} + \Delta \boldsymbol{x}$ ；
- 5) 判断总体误差是否收敛，如果收敛 $\boldsymbol{x}^* = \boldsymbol{x}$ 。不收敛时，如果总体误差减小则 $\lambda \leftarrow \frac{\lambda}{2}$ ；如果总体误差增大则 $\lambda \leftarrow 2\lambda$ ，返回第一步；

4.4 仿真实验

4.4.1 后端优化算法实验

为了验证本文后端优化算法的有效性，使用公开的数据集进行实验，使用本文 SLAM 算法（包括激光里程计和后端优化）与本文激光里程计算法进行对比。选择 Magazion 机器人录制的公开数据集进行实验，数据集有 2D 激光雷达数据、IMU 数据和里程计数据，其中激光雷达的频率为 15Hz，IMU 的频率为 250Hz，里程计的频率为 125Hz，数据集的具体信息如图 4.4 所示。

```
path: hw2.bag
version: 2.0
duration: 5:50s (350s)
start: Dec 18 2017 05:03:12.52 (1513602192.52)
end: Dec 18 2017 05:09:03.48 (1513602543.48)
size: 102.8 MB
messages: 201263
compression: none [134/134 chunks]
types:
  nav_msgs/Odometry [cd5e73d190d741a2f92e81eda573aca7]
  sensor_msgs/Imu [6a62c6daae103f4ff57a132d6f95cec2]
  sensor_msgs/LaserScan [90c7ef2dc6895d81024acba2ac42f369]
  tf2_msgs/TFMessage [94810edda583a504dfda3829e70d7eec]
topics:
  /imu/data_raw_transformed 87491 msgs : sensor_msgs/Imu
  /odom 35093 msgs : nav_msgs/Odometry
  /scan_front 4455 msgs : sensor_msgs/LaserScan
  /scan_rear 4455 msgs : sensor_msgs/LaserScan
  /tf 69767 msgs : tf2_msgs/TFMessage
  /tf_static 2 msgs : tf2_msgs/TFMessage
```

图 4.4 数据集的详细信息

图 4.5 和图 4.6 分别是本文激光里程计算法和 SLAM 算法构建的地图，可以看出激光里程计算法构建的地图墙体发生了明显倾斜，内部存在重影，边界模糊，地图严重变形失真；而 SLAM 算法构建的地图墙体没有明显倾斜，内部不存在重影，轮廓和细节都十分清晰，地图特征明显；可见本文 SLAM 算法的建图效果明显高于本文激光里程计算法的建图效果。

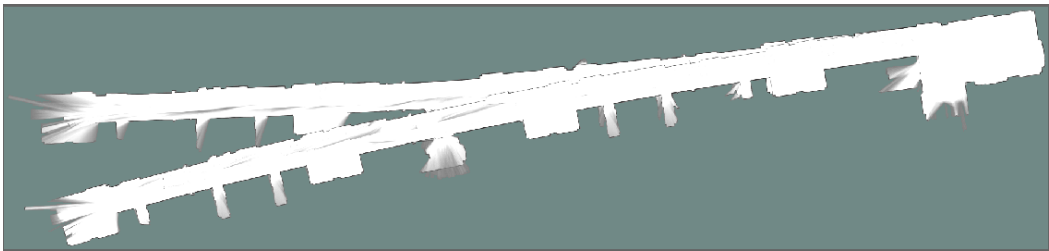


图 4.5 本文激光里程计算法构建的地图



图 4.6 本文 SLAM 算法构建的地图

为了比较两种算法的定位精度，将两种算法的估计轨迹与真实轨迹进行对比分析，不同算法的估计轨迹对比如图 4.7 所示，图中黑色实线为机器人的真实轨迹，红色点划线为本文 SLAM 算法的估计轨迹，蓝色虚线为本文激光里程计算法的估计轨迹，可以发现 SLAM 算法的估计轨迹比激光里程计算法的估计轨迹更接近真实轨迹。

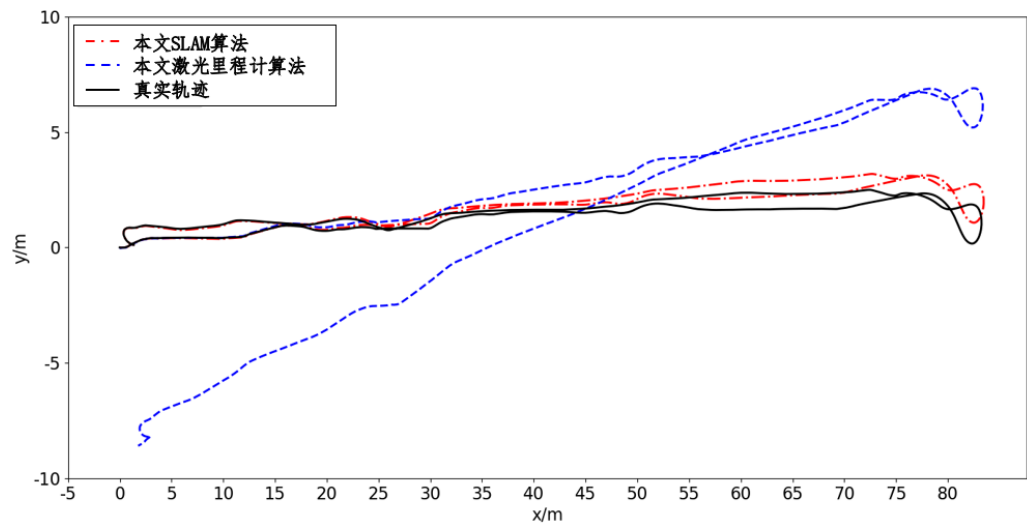


图 4.7 不同算法估计轨迹对比

然后使用 evo 计算绝对轨迹误差，比较不同算法估计轨迹的误差，结果如表 4.1 所示。本文 SLAM 算法在均方根误差、误差平方和以及标准差均小于本文激光里程计算法，表明本文 SLAM 算法的估计轨迹，比本文激光里程计算法的估计轨迹更接近真实轨迹，可见本文 SLAM 算法的定位精度高于本文激光里程计算法。

表 4.1 绝对轨迹误差

算法	均方根误差 (m)	误差平方和 (m ²)	标准差 (m)
SLAM 算法	0.658840	1062.603961	0.360500
激光里程计算法	3.491622	29844.604516	2.241245

通过上述的实验说明了 SLAM 算法的建图效果和定位精度均高于激光里程计算法，其原因是 SLAM 算法使用后端优化算法，而激光里程计算法未使用后端优化算法。可见使用后端优化算法可以提高 SLAM 算法的建图效果和定位精度，同时也证明了本文后端优化算法的有效性。

4.4.2 闭环检测算法实验

为了证明本文的闭环检测算法可以快速检测出闭环，与使用 Cartographer 闭环检测方案的 SLAM 算法(以下简称对照 SLAM 算法)进行对比，其中对照 SLAM 算法，除闭环检测算法外，其他部分与本文的 SLAM 算法保持一致。实验的硬件设备为 PC 机，其

内存为 8GB RAM, CPU 型号为 Intel Core i7-9750, 操作系统为 Ubuntu18.04, 实验环境为 ROS 系统。本文设计了两组实验来比较本文 SLAM 算法和对照 SLAM 算法的性能, 第一组实验采用小场景数据集进行实验, 第二组实验采用大场景数据集进行实验。

1) 小场景实验

本实验使用图 4.4 所示的数据集进行实验, 图 4.8 是对照 SLAM 算法构建的地图, 该地图不存在重影, 轮廓和细节十分清晰, 特征明显。与本文 SLAM 算法构建的地图对比发现, 本文 SLAM 算法和对照 SLAM 算法在建图效果上没有明显差别。



图 4.8 对照 SLAM 算法构建的地图

为了比较两种 SLAM 算法的性能, 对两种 SLAM 算法的 CPU 使用率进行了统计, 两种 SLAM 算法 CPU 使用率对比如图 4.9 所示, 可以看出本文 SLAM 算法的 CPU 使用率略低于对照 SLAM 算法。通过对统计数据分析发现, 本文 SLAM 算法的运行时间为 358 s, 对照 SLAM 算法的运行时间为 360 s, 本文 SLAM 算法的运行时间相比对照 SLAM 算法缩短了 0.6%; 其中本文 SLAM 算法的 CPU 的平均使用率为 38.8%, 对照 SLAM 算法 CPU 的平均使用率为 39.1%, 本文 SLAM 算法 CPU 平均使用率相比对照 SLAM 算法降低了 0.76%; 可见在小场景的建图实验中, 本文 SLAM 算法的性能优于对照 SLAM 算法的性能。

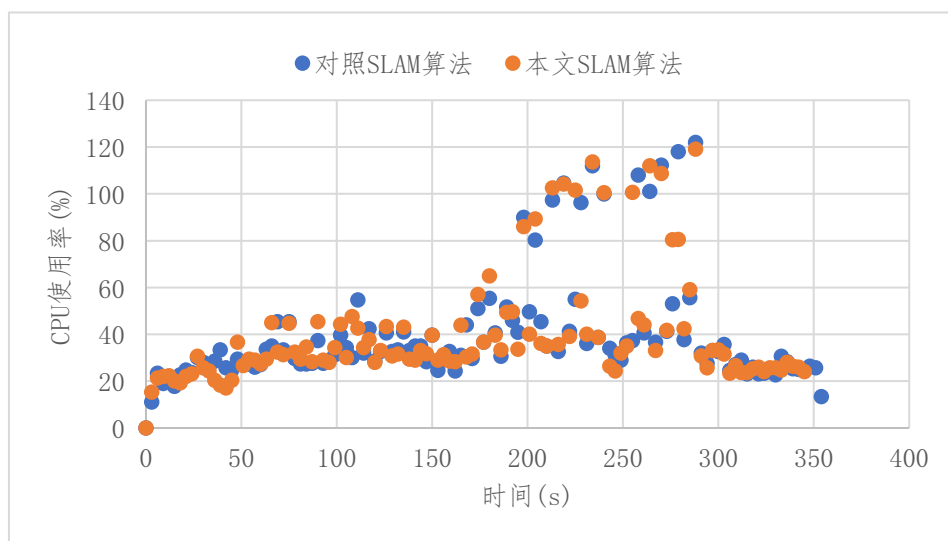


图 4.9 不同 SLAM 算法 CPU 使用率对比

对 SLAM 算法闭环检测的时长进行统计分析，其中本文 SLAM 算法的闭环检测时长为 24915 ms，对照 SLAM 算法的闭环检测时长为 31539 ms，本文 SLAM 算法的闭环检测时长相比对照 SLAM 算法缩短了 21%。表明在小场景的建图实验中，本文 SLAM 算法的闭环检测速度快于对照 SLAM 算法。

2) 大场景实验

大场景的实验选取德意志博物馆的数据集进行实验^[27]，该数据集的时长为 1912s，数据集包含 2D 激光雷达数据、IMU 数据，其中激光雷达的频率为 35Hz，IMU 的频率为 250Hz，数据集的具体信息如图 4.10 所示。

```
path:      cartographer_paper_deutsches_museum.bag
version:   2.0
duration:  31:52s (1912s)
start:     May 26 2015 06:30:16.48 (1432647016.48)
end:       May 26 2015 07:02:09.46 (1432648929.46)
size:      470.5 MB
messages:  617965
compression: bz2 [3334/3334 chunks; 18.31%]
uncompressed: 2.5 GB @ 1.3 MB/s
compressed:  462.3 MB @ 247.5 KB/s (18.31%)
types:      sensor_msgs/Imu [6a62c6daae103f4ff57a132d6f95cec2]
            sensor_msgs/MultiEchoLaserScan [6fefb0c6da89d7c8abe4b339f5c2f8fb]
topics:     horizontal_laser_2d 70358 msgs : sensor_msgs/MultiEchoLaserScan
            imu                  478244 msgs : sensor_msgs/Imu
            vertical_laser_2d    69363 msgs : sensor_msgs/MultiEchoLaserScan
```

图 4.10 德意志博物馆数据集

图 4.11 是本文 SLAM 算法构建的地图，该地图存在重影，边界存在毛刺，但是整体来看场景特征完整，不存在形变失真情况；图 4.12 是对照 SLAM 算法构建的地图，该地图也存在重影和毛刺，但是整体来看场景特征完整，不存在形变失真情况。可见在大场景的建图实验中，本文 SLAM 算法和对照 SLAM 算法在建图效果上没有明显差别。

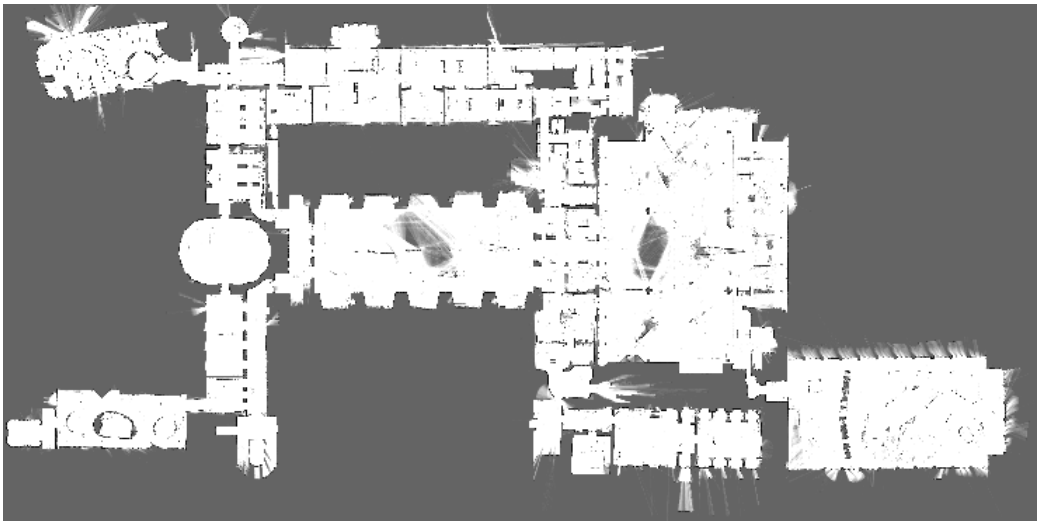


图 4.11 本文 SLAM 算法构建的地图

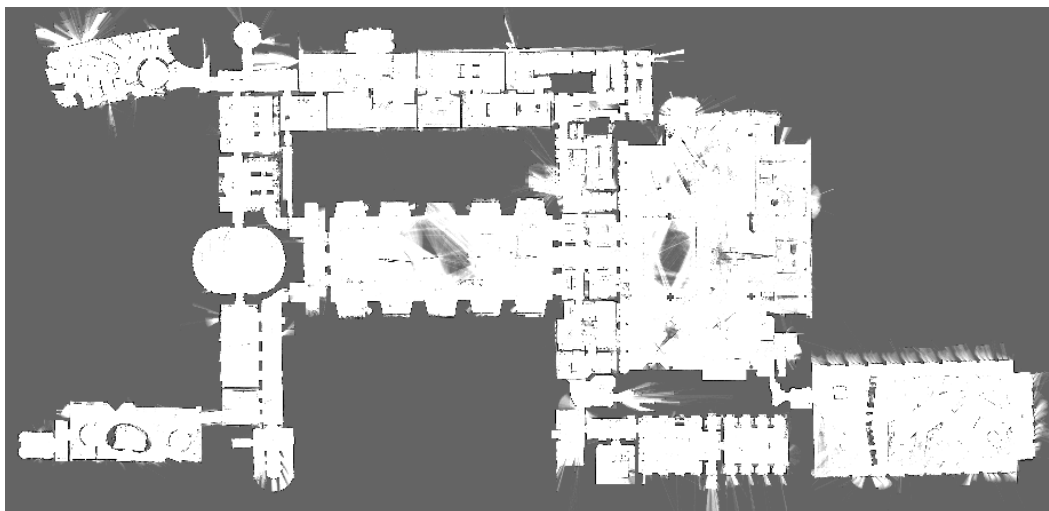


图 4.12 对照 SLAM 算法构建的地图

接着,对两种 SLAM 算法 CPU 的使用率进行了统计,两种 SLAM 算法 CPU 使用率对比如图 4.13 所示,可以看出本文 SLAM 算法 CPU 使用率明显低于对照 SLAM 算法。通过对统计数据计算发现,本文 SLAM 算法的运行时间为 1920 s,对照 SLAM 算法的运行时间为 2541 s,本文 SLAM 算法的运行时间相比对照 SLAM 算法缩短了 24%;本文 SLAM 算法的 CPU 的平均使用率为 254%,对照 SLAM 算法 CPU 的平均使用率为 271%,本文 SLAM 算法 CPU 平均使用率相比对照 SLAM 算法降低了 6.3%。可见在大场景的建图实验中,本文 SLAM 算法的性能优于对照 SLAM 算法的性能。

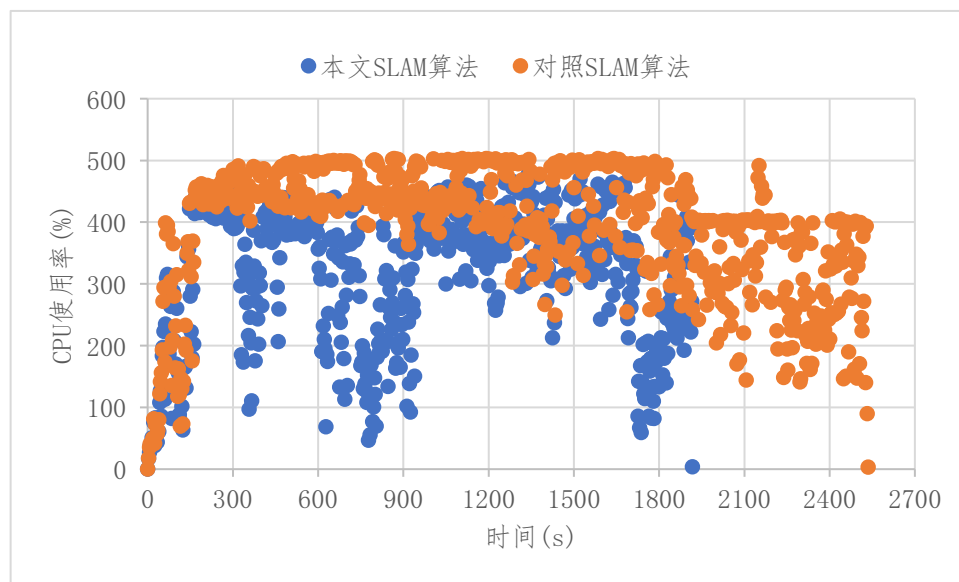


图 4.13 不同 SLAM 算法 CPU 使用率对比

最后对 SLAM 算法闭环检测的时长进行统计分析,其中本文 SLAM 算法的闭环检测时长为 207592 ms,对照 SLAM 算法的闭环检测时长为 1109156 ms,本文 SLAM 算

法的闭环检测时长相比对照 SLAM 算法缩短了 81%，可见在大场景的建图实验中，本文 SLAM 算法的闭环检测速度快于对照 SLAM 算法。

在两组实验中，无论场景大小，两种 SLAM 算法的建图效果没有明显区别，说明本文的闭环检测算法可以保证 SLAM 算法的建图效果。此外，使用本文闭环检测方案的 SLAM 算法 CPU 的使用率会降低，闭环检测时长会缩短，表明本文的闭环检测方案相对于 Cartographer 的闭环检测方案更能保证 SLAM 系统的实时性。

4.5 本章小结

本章研究了基于图优化的后端优化算法，首先采用像素匹配进行闭环检测，使用分支定界加速闭环检测；然后根据检测到的闭环构建位姿图，建立目标函数；最后采用列文伯格-马夸尔特法对目标函数进行求解。相较于传统的闭环检测算法，本文的闭环检测算法更能保证 SLAM 算法的实时性。通过后端优化算法仿真实验，证明了后端优化算法的有效性；与对照 SLAM 算法相比，在小场景的建图实验中，本文 SLAM 算法消耗的计算资源降低了 0.76%，闭环检测时长缩短了 21%；在大场景的建图实验中，本文 SLAM 算法消耗的计算资源降低了 6.3%，闭环检测时长缩短了 81%，提高了闭环检测算法的实时性。

5 实验与分析

本章搭建了实验平台并开展了实验研究，在室内环境开展了建图实验，通过比较本文激光里程计算法与 Hector 算法的建图效果、建图精度，证明了本文激光里程计算法的有效性；在室内环境和长廊环境分别开展了建图实验，通过比较本文 SLAM 算法与 Cartographer 算法的建图效果、建图精度以及算法实时性，证明了本文 SLAM 算法的有效性。

5.1 实验平台搭建

根据三维模型对移动机器人进行装配，移动机器人样机如图 5.1 所示。图 5.1（a）所示为样机的外部结构，包括底盘、外壳、负载板、车身等，车身装有启动按钮、充电接口、充电刷块、数据接口、急停按钮、防撞条等装置。图 5.1（b）所示为样机的内部结构，内部安装有驱动电机、减振结构、工控机、嵌入式控制器、电池等设备，其中嵌入式控制器安装在工控机的下方，提高了内部空间利用率。

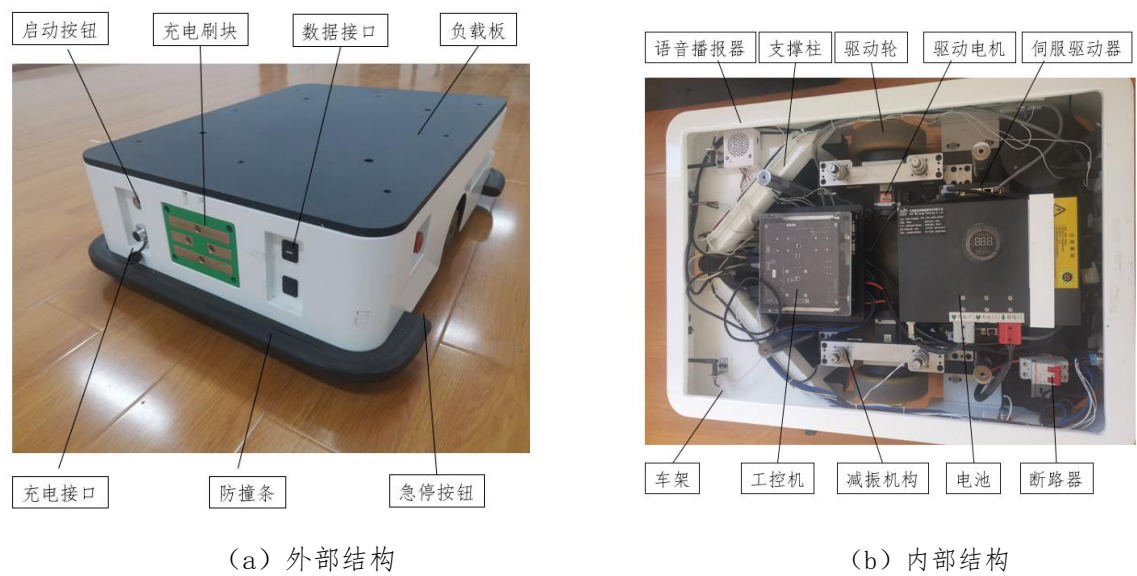


图 5.1 移动机器人样机

在移动机器人运行的过程中，使用 Rviz 将传感器数据、SLAM 算法的定位和建图的结果实时展示，上位机使用 Rviz 订阅 SLAM 算法发布的话题，就可以实时展示 SLAM 算法构建的地图，图 5.2 为地图的显示界面。根据要求设计了机器人的运动控制程序，该

程序根据输入的命令，对机器人进行运动控制，如图 5.3 所示为运动控制界面，通过键盘输入指令控制机器人移动。

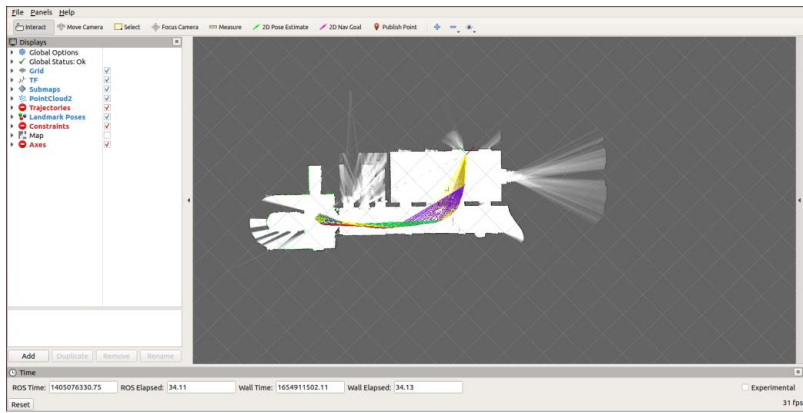


图 5.2 SLAM 系统地图显示界面

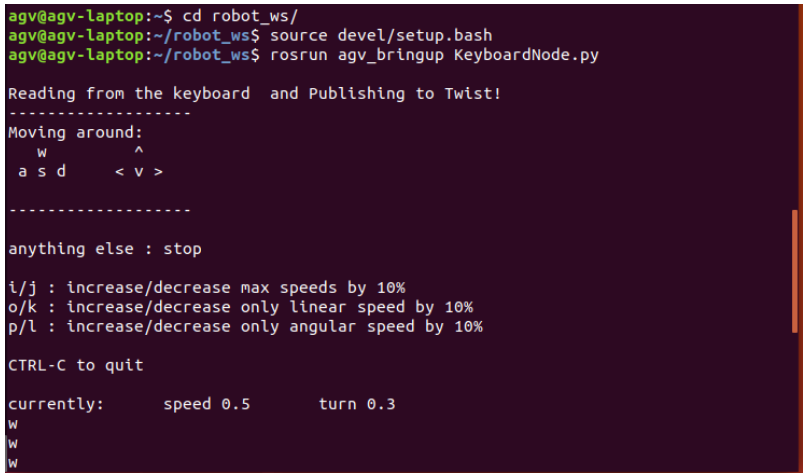


图 5.3 运动控制界面

5.2 激光里程计的对比实验

5.2.1 实验方案

为了验证本文激光里程计算法有效性，使用本文的激光里程计算法与 Hecror 算法进行对比，本文将对两种算法的建图效果和建图精度进行分析。

通过比较算法构建的地图是否完整、地图特征是否清晰、边界是否连续，来比较激光里程计算法的建图效果；为了对算法的建图精度进行比较，通过计算激光里程计算法建图得到环境特征长度的相对误差和绝对误差，可以通过数据准确反映建图精度。本文设计了实验来比较本文激光里程计算法和 Hecror 算法的性能，实验场景为实验室的室内环境，如图 5.4 所示。



图 5.4 实验室的室内场景

为了保证两种激光里程计算法具有完全相同的数据输入，通过上位机控制机器人在实验环境中移动，其中激光雷达的参数配置如图 5.5 所示，使用 ROS 中相关命令录制数据集，该数据集包含激光雷达、IMU、里程计的数据，数据集录制如图 5.6 所示。

```
<node name="PavoScanNode" pkg="pavo_ros" type="pavo_scan_node">
  <remap from="/PavoScanNode/scan" to="scan" />
  <param name="frame_id" type="string" value="laser"/>
  <param name="scan_topic" type="string" value="scan" />
  <param name="angle_min" type="double" value="-90.00" /> <!--扫描角度 -135至135-->
  <param name="angle_max" type="double" value="90.00"/>
  <param name="range_min" type="double" value="0.10" />
  <param name="range_max" type="double" value="15.0" /> <!--扫描距离, 最大20, -->
  <param name="inverted" type="bool" value="false"/>
  <param name="enable_motor" type="bool" value="$(arg enable_motor)"/>
  <param name="motor_speed" type="int" value="15" />
  <param name="merge_coef" type="int" value="2" />
  <param name="lidar_ip" type="string" value="10.10.10.101" />
  <param name="lidar_port" type="int" value="2368" />
  <param name="method" type="int" value="$(arg method)" />
  <param name="switch_active_mode" type="bool" value="false"/>
</node>
```

图 5.5 激光雷达的参数配置

```
wangkale@ubuntu:~/BAG$ rosbag record imu scan odom tf
[ INFO] [1647070553.740813208]: Subscribing to imu
[ INFO] [1647070553.744228735]: Subscribing to odom
[ INFO] [1647070553.747306847]: Subscribing to scan
[ INFO] [1647070553.749489329]: Subscribing to tf
[ INFO] [1647070553.752247758]: Recording to '2022-03-11-23-35-53.bag'.
```

图 5.6 数据集录制

激光里程计的对比实验使用上位机控制机器人在实验室环境移动，同时录制数据集，为了保证建图效果，控制机器人在环境中移动两圈，分别使用本文的激光里程计算法和 Hecror 算法进行建图，根据构建的地图比较算法的定位效果和建图精度。

5.2.2 实验结果与分析

图 5.7 是本文激光里程计算法构建的地图，该地图边界模糊，场景特征完整，不存在形变失真情况；图 5.8 是 Hector 算法构建的地图，该地图边界模糊，场景特征完整，不存在形变失真情况，可见本文激光里程计算法和 Hector 算法在建图效果上没有明显差别。



图 5.7 本文激光里程计算法构建的地图



图 5.8 Hector 算法构建的地图

为了比较激光里程计算法的建图精度，在实验场景中选取四个典型的特征，通过测量获取特征的真实长度，计算算法建图得到环境特征长度的相对误差和绝对误差，特征的分布如图 5.9 所示。本文激光里程计算法的测量数据和误差值如表 5.1 所示，其中最大绝对误差为 61 mm，最小的绝对误差为 16 mm；最大的相对误差为 1.20%，最小的相对误差为 0.70%；平均相对误差为 0.91%；Hector 算法的测量数据和误差值如表 5.2 所示，其中最大的绝对误差为 67 mm，最小的绝对误差为 21 mm；其中最大的相对误差为 1.31%，最小的相对误差为 0.78%；平均相对误差为 1.00%。

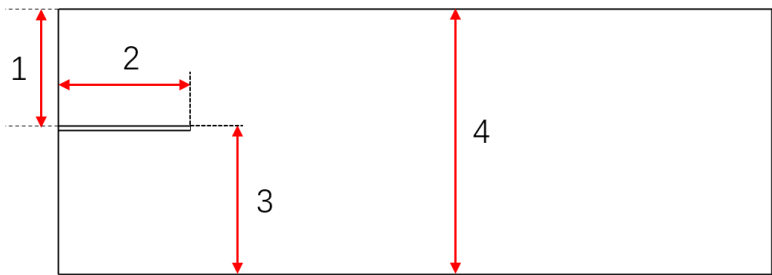


图 5.9 特征分布

表 5.1 本文激光里程计算法的测量数据

特征点的编号	真实长度(mm)	测量长度(mm)	绝对误差(mm)	相对误差(%)
1	2274	2258	16	0.70
2	2597	2572	25	0.96
3	2813	2791	22	0.78
4	5087	5026	61	1.20
平均	-	-	-	0.91

表 5.2 Hector 算法的测量数据

测量点的编号	真实长度(mm)	测量长度(mm)	绝对误差(mm)	相对误差(%)
1	2274	2253	21	0.92
2	2597	2571	26	1.00
3	2813	2791	22	0.78
4	5087	5020	67	1.31
平均	-	-	-	1.00

如图 5.10 所示，为两种算法的相对误差曲线，在 4 个特征点上本文激光里程计的相对误差均小于 Hector 算法；并且本文激光里程计算法的平均相对误差相比 Hector 算法降低了 9.0%，表明在激光里程计的对比实验中，本文激光里程计算法的建图精度高于 Hector 算法。

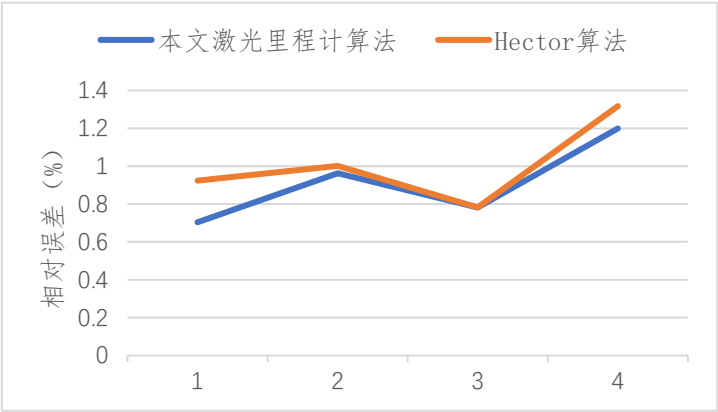


图 5.10 不同算法的相对误差对比

综上，激光里程计对比实验中，两种算法的建图效果没有明显区别，但本文激光里程计算法的建图精度高于 Hector 算法，可以证明本文激光里程计算法的有效性。

5.3 SLAM 算法的对比实验

5.3.1 实验方案

为了验证本文 SLAM 算法的有效性，使用本文 SLAM 算法与 Cartographer 算法进行对比。为了比较 SLAM 算法性能的优劣，将对两种 SLAM 算法的建图效果、建图精度、

算法实时性进行对比分析。为了比较 SLAM 算法实时性,使用脚本监测 SLAM 算法的 CPU 使用率,通过比较 CPU 使用率来比较两种 SLAM 算法的实时性。

本文设计了两组实验来比较本文 SLAM 算法和 Cartographer 算法的性能,第一组实验的场景为实验室的室内场景,如图 5.11 所示;第二组实验场景为实验室外的长廊环境,如图 5.12 所示。



图 5.11 实验室的室内环境



图 5.12 长廊环境

SLAM 算法的对比实验使用上位机控制机器人在实验室环境移动,同时录制数据集,分别使用本文的 SLAM 算法和 Cartographer 算法进行建图,根据构建的地图比较算法的定位效果和建图精度,根据脚本采集的 CPU 使用率对算法的实时性进行比较。

5.3.2 实验结果与分析

1) 室内场景的建图实验

图 5.13 是本文 SLAM 算法构建的地图,该地图边界连续,不存在毛刺,场景特征完整,不存在形变失真情况;图 5.14 是 Cartographer 算法构建的地图,该地图边界连续,不存在毛刺,场景特征完整,不存在形变失真情况。可见在室内场景的建图实验中,本文 SLAM 算法和 Cartographer 算法在建图效果上没有明显差别。

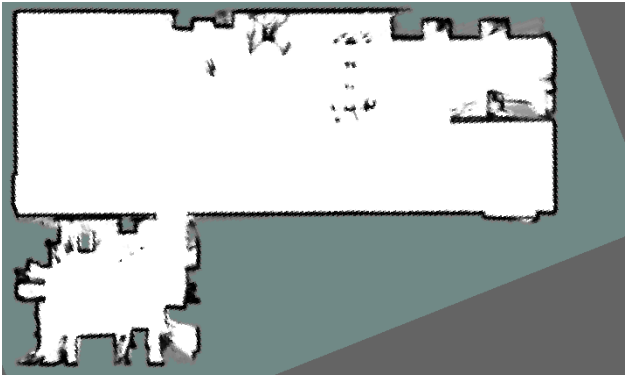


图 5.13 本文 SLAM 算法构建的地图



图 5.14 Cartographer 算法构建的地图

为了比较 SLAM 算法的建图精度，在实验场景中选取七个典型的特征，通过测量获取特征的真实长度，计算 SLAM 算法建图得到环境特征长度的相对误差和绝对误差，特征分布如图 5.15 所示。本文 SLAM 算法的测量数据和误差值如表 5.3 所示，其中最大绝对误差为 81 mm，最小的绝对误差为 10 mm；最大的相对误差为 2.15%，最小的相对误差为 0.38%；平均相对误差为 1.30%；Cartographer 算法的测量数据和误差值如表 5.4 所示，其中最大的绝对误差为 88 mm，最小的绝对误差为 12 mm；其中最大的相对误差为 2.23%，最小的相对误差为 0.46%；平均相对误差为 1.41%。

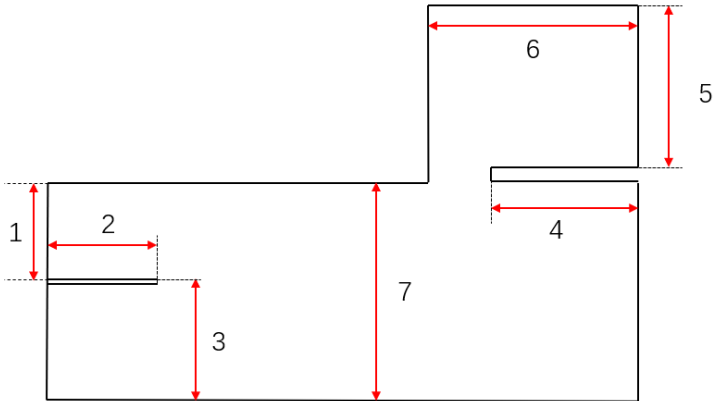


图 5.15 特征分布

表 5.3 本文 SLAM 算法的测量数据

特征点的编号	真实长度(mm)	测量长度(mm)	绝对误差(mm)	相对误差(%)
1	2274	2262	12	0.52
2	2597	2587	10	0.38
3	2813	2840	27	0.95
4	3528	3452	76	2.15
5	3674	3609	65	1.76
6	4605	4524	81	1.75
7	5087	5008	79	1.55
平均	-	-	-	1.30

表 5.4 Cartographer 算法的测量数据

测量点的编号	真实长度(mm)	测量长度(mm)	绝对误差(mm)	相对误差(%)
1	2274	2289	15	0.65
2	2597	2609	12	0.46
3	2813	2782	31	1.10
4	3528	3449	79	2.23
5	3674	3606	68	1.85
6	4605	4693	88	1.91
7	5087	5001	86	1.70
平均	-	-	-	1.41

如图 5.16 所示，为两种 SLAM 算法的相对误差曲线，在 7 个特征点上本文 SLAM 算法的相对误差均小于 Cartographer 算法；并且本文 SLAM 算法的平均相对误差相比 Cartographer 算法降低了 7.80%，表明在室内场景的建图实验中，本文 SLAM 算法的建图精度高于 Cartographer 算法。

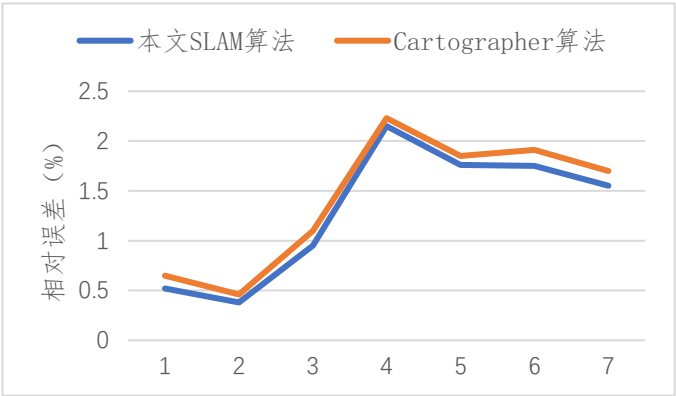


图 5.16 不同 SLAM 算法的相对误差对比

对两种 SLAM 算法的 CPU 使用率进行了分析，两种 SLAM 算法 CPU 使用率对比如图 5.17 所示，可以看出本文 SLAM 算法的 CPU 使用率略低于 Cartographer 算法。通过对统计数据分析发现，本文 SLAM 算法的运行时间为 192 s，Cartographer 算法的运行时间为 193 s，本文 SLAM 算法和 Cartographer 算法的运行时间几乎相同；其中本文 SLAM 算法的 CPU 的平均使用率为 22.1%，Cartographer 算法 CPU 的平均使用率为 25.3%，本文

SLAM 算法 CPU 平均使用率相比 Cartographer 算法降低了 12.6%；可见在室内环境的建图实验中，本文 SLAM 算法的实时性优于 Cartographer 算法。

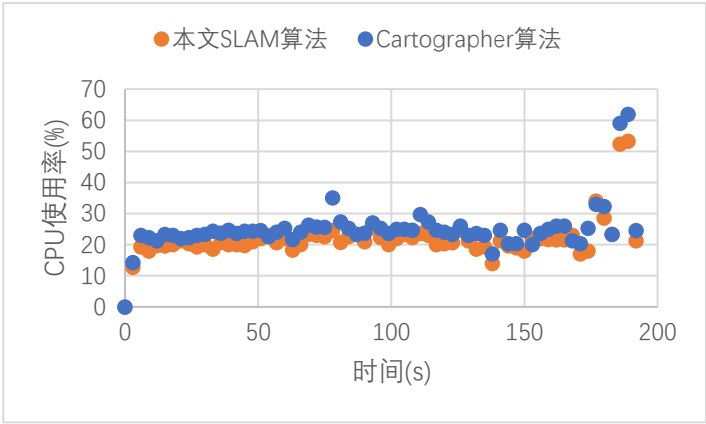


图 5.17 不同 SLAM 算法 CPU 使用率对比

2) 长廊的建图实验

图 5.18 是本文 SLAM 算法构建的地图，该地图存在重影，边界存在毛刺且不连续，但是整体来看场景特征完整，不存在严重形变失真情况；图 5.19 是 Cartographer 算法构建的地图，该地图存在重影，边界有毛刺且不连续，但是场景特征完整，不存在严重形变失真情况。表明在长廊的建图实验中，本文 SLAM 算法和 Cartographer 算法在建图效果上没有明显差别。

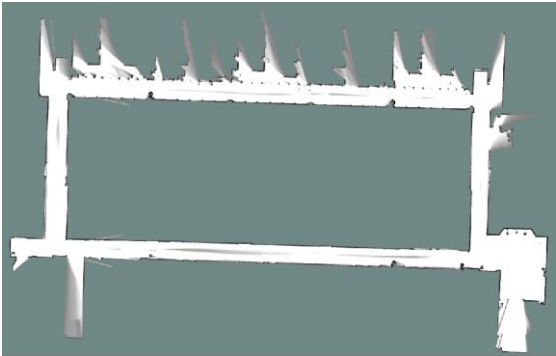


图 5.18 本文 SLAM 算法构建的长廊地图

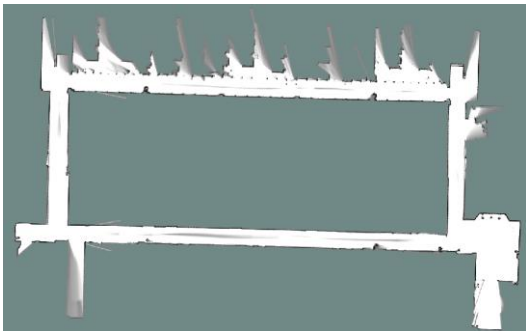


图 5.19 Cartographer 算法构建的长廊地图

然后，在实验场景中选取六个典型的特征，通过测量获取特征的实际长度，计算 SLAM 算法建图得到环境特征长度的相对误差和绝对误差，特征分布如图 5.20 所示。本文 SLAM 算法测量数据和误差值如表 5.5 所示，其中最大绝对误差为 1438 mm，最小的绝对误差为 9 mm；最大的相对误差为 3.16%，最小的相对误差为 0.58%，平均相对误差为 1.63%；Cartographer 算法测量数据和误差值如表 5.6 所示，其中最小的绝对误差为 13 mm，最大的绝对误差为 1497 mm；其中最大的相对误差为 3.29%，最小的相对误差为 0.69%，平均相对误差为 1.87%。

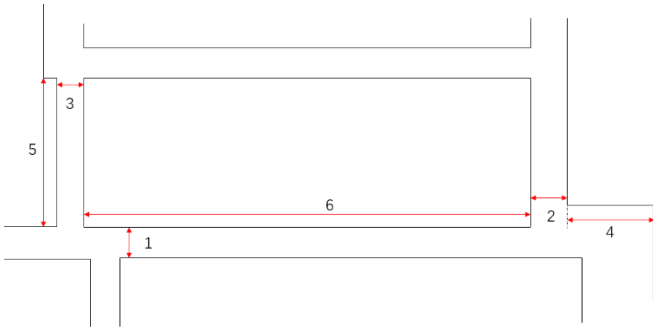


图 5.20 特征分布

表 5.5 本文 SLAM 算法的测量数据

测量点的编号	真实值(mm)	测量值(mm)	绝对误差(mm)	相对误差(%)
1	1945	1932	13	0.67
2	1897	1886	11	0.58
3	1996	1987	9	0.45
4	6868	7015	147	2.14
5	16120	15672	448	2.78
6	45535	44097	1438	3.16
平均	-	-	-	1.63

表 5.6 Cartographer 算法的测量数据

测量点的编号	真实值(mm)	测量值(mm)	绝对误差(mm)	相对误差(%)
1	1945	1929	16	0.82
2	1897	1884	13	0.69
3	1996	2040	44	2.20
4	6868	7065	197	2.87
5	16120	16644	524	3.25
6	45535	44038	1497	3.29
平均	-	-	-	1.87

如图 5.21 所示，为两种 SLAM 算法的相对误差曲线，可以发现在六个特征点上本文 SLAM 算法的相对误差均小于 Cartographer 算法；并且本文 SLAM 算法的平均相对误差相比 Cartographer 算法降低了 12.8%，表明在长廊的建图实验中，本文 SLAM 算法的建图精度高于 Cartographer 算法。

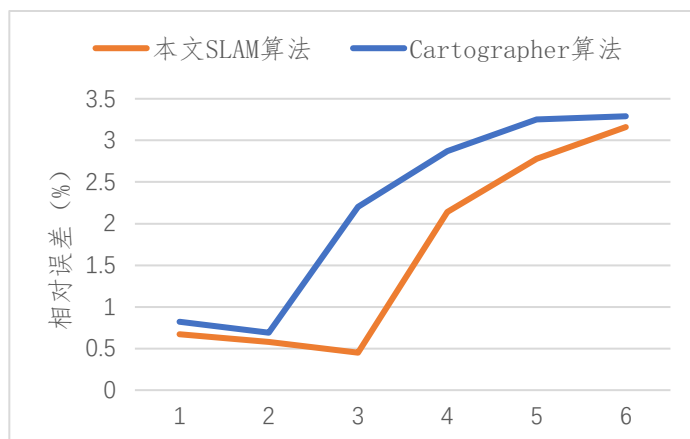


图 5.21 不同 SLAM 算法的相对误差对比

然后，对两种 SLAM 算法的 CPU 使用率进行了统计，两种 SLAM 算法 CPU 使用率对比如图 5.22 所示，可以看出本文 SLAM 算法的 CPU 使用率略低于 Cartographer 算法。通过对统计数据分析发现，本文 SLAM 算法的运行时间为 1029 s，Cartographer 算法的运行时间为 1030 s，两种 SLAM 算法的运行时间几乎相同；其中本文 SLAM 算法的 CPU 的平均使用率为 34.3%，Cartographer 算法 CPU 的平均使用率为 29.4%，本文 SLAM 算法 CPU 平均使用率相比 Cartographer 算法降低了 14.3%。可见在长廊的建图实验中，本文 SLAM 算法的性能优于 Cartographer 算法。

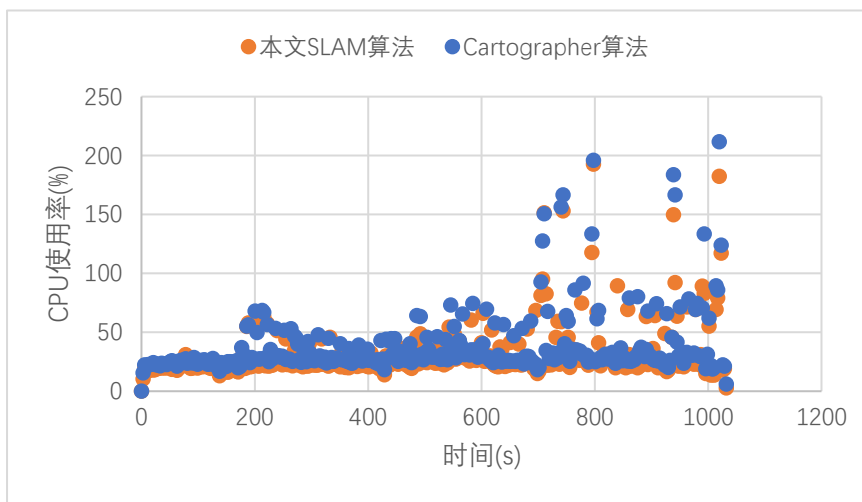


图 5.22 不同 SLAM 算法 CPU 使用率对比

综上，在两组实验中，两种 SLAM 算法的建图效果没有明显区别，但本文 SLAM 算法的建图精度高于 Cartographer 算法，并且本文 SLAM 算法的 CPU 平均使用率低于 Cartographer 算法，表明本文 SLAM 算法的建图精度和实时性高于 Cartographer 算法，可以证明本文 SLAM 算法的有效性。

5.4 本章小结

本章搭建了双轮差速移动机器人的实验样机，在室内环境开展了建图实验，比较了本文激光里程计算法与 HECTOR 算法的建图效果、建图精度，本文激光里程计算法的平均相对误差降低了 9.1%，证明了本文激光里程计算法的有效性。在室内环境和长廊环境分别开展了建图实验，比较了本文 SLAM 算法与 Cartographer 算法的建图效果、建图精度以及算法实时性；在室内建图实验中，本文 SLAM 算法的平均相对误差降低了 7.8%，系统消耗的计算资源降低了 12.6%；在长廊建图实验中，本文 SLAM 算法的平均相对误差降低了 12.8%，系统消耗的计算资源降低了 14.3%，证明了本文 SLAM 算法的有效性。

6 总结与展望

6.1 全文总结

本文面向移动机器人定位和建图需求，研究了激光 SLAM 前端激光里程计和后端优化算法，设计并搭建了双轮差速移动机器人实验平台，研究内容主要包括以下几个部分：

(1) 设计了移动机器人实验平台的硬件和软件系统。首先，给出了硬件系统总体方案，对机械模块、控制模块、感知模块和驱动模块进行详细设计，对关键元器件行选型；然后，设计了软件系统的整体框，基于 ROS 搭建了上位机软件系统，对其中的定位与建图、传感器数据处理、运动规划等模块进行了设计；最后，给出了下位机嵌入式软件系统的整体功能，对传感器数据采集、运动控制等模块进行了设计。

(2) 研究了 SLAM 激光里程计算法，实现了移动机器人的位姿估计和子图构建。首先使用 IMU 预积分对激光点云进行畸变矫正；然后使用差速驱动机器人的位姿估计算法对机器人的位姿进行估计，并为帧与图扫描匹配算法提供初值；最后通过启发式方法实现了关键帧的选取，根据关键帧进行子图构建。通过对比本文激光里程计算法与 Hector 算法的建图效果和定位精度，证明了本文激光里程计算法的有效性；与使用 LEGO-LOAM 位姿估计方案的激光里程计算法进行对比，本文激光里程计估计轨迹与真实轨迹的相对误差下降了 25%。

(3) 设计了 SLAM 后端优化算法，减小了前端激光里程计的累计误差，提高了算法的建图性能。通过像素匹配进行闭环检测，使用分支定界加速闭环检测，提高了闭环检测的速度；根据检测到的闭环构建位姿图，建立目标函数，采用列文伯格-马夸尔特法进行求解，实现了位姿图优化。通过比较激光里程计算法与 SLAM 算法的建图效果和定位精度，证明了本文后端优化算法的有效性。与对照 SLAM 算法相比，在小场景的建图实验中，本文 SLAM 算法消耗的计算资源降低了 0.76%，闭环检测时长缩短了 21%；在大场景的建图实验中，本文 SLAM 算法消耗的计算资源降低了 6.3%，闭环检测时长缩短了 81%。

(4) 搭建实验平台并开展了实验研究。为验证本文激光里程计算法的有效性,在室内环境开展了建图实验,与开源 HECTOR 算法相比,本文激光里程计算法的平均相对误差降低了 9.1%。为验证本文 SLAM 算法的有效性和建图性能,在室内环境和长廊环境分别开展了建图实验,与开源 Cartographer 算法相比,在室内建图实验中,本文 SLAM 算法的平均相对误差降低了 7.8%,系统消耗的计算资源降低了 12.6%;在长廊建图实验中,本文 SLAM 算法的平均相对误差降低了 12.8%,系统消耗的计算资源降低了 14.3%。

6.2 工作展望

本文设计并搭建了移动机器人实验平台,研究了 SLAM 激光里程计算法及后端优化算法,提高了移动机器人的定位和建图性能。但论文目前依然存在一些不足之处,后续可以在以下几方面进一步研究和改进:

(1) 融合视觉、激光等多源异构传感器信息,研究基于多传感器融合的激光 SLAM 算法,提升移动机器人定位建图的精度及鲁棒性。

(2) 从特征点、深度估计、位姿估计、重定位等方面出发,研究与深度学习相结合的激光 SLAM 算法,利用深度学习取代传统定位和建图的部分环节,提升 SLAM 系统的定位和建图性能。

(3) 针对大范围复杂环境的场景感知需求,研究多机器人协同 SLAM 算法,实现多个机器人在协同工作时的共同定位和地图构建。

参考文献

- [1] 陈浙明, 张海宇, 马青. 物流设备自动化技术发展趋势 [J]. 起重运输机械, 2021(S1): 41-43.
- [2] Le-Anh T, De Koster M. A review of design and control of automated guided vehicle systems [J]. European Journal of Operational Research, 2006, 171(1): 1-23.
- [3] 高翔. 视觉 SLAM 十四讲: 从理论到实践 [M]. 北京: 电子工业出版社, 2017.
- [4] Smith R C, Cheeseman P. On the representation and estimation of spatial uncertainty [J]. The international journal of Robotics Research, 1986, 5(4): 56-68.
- [5] Leonard J J, Durrant-Whyte H F. Mobile robot localization by tracking geometric beacons [J]. IEEE Transactions on robotics and Automation, 1991, 7(3): 376-382.
- [6] Moutarlier P, Chatila R. Stochastic Multisensory Data Fusion for Mobile Robot Location and Environment Modelling[C]. Proceedings of ISER, 1989.
- [7] Julier S J, Uhlmann J K. Unscented filtering and nonlinear estimation [J]. Proceedings of the IEEE, 2004, 92(3): 401-422.
- [8] Jiang X, Li T, Yu Y. A novel SLAM algorithm with Adaptive Kalman filter[C]. 2016 International Conference on Advanced Robotics and Mechatronics (ICARM), 2016: 107-111.
- [9] Tardós J D, Neira J, Newman P M, et al. Robust mapping and localization in indoor environments using sonar data [J]. The International Journal of Robotics Research, 2002, 21(4): 311-330.
- [10] Folkesson J, Christensen H. Graphical SLAM-a self-correcting map[C]. IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004, 2004: 383-390.
- [11] Bosse M, Newman P, Leonard J, et al. An atlas framework for scalable mapping[C]. 2003 IEEE International Conference on Robotics and Automation, 2003: 1899-1906.
- [12] Hammersley J M, Morton K W. Poor man's monte carlo [J]. Journal of the Royal Statistical Society: Series B (Methodological), 1954, 16(1): 23-38.
- [13] Del Moral P. Nonlinear filtering: Interacting particle resolution [J]. Comptes Rendus de l'Académie des Sciences-Series I-Mathematics, 1997, 325(6): 653-658.
- [14] Murphy K, Russell S: Rao-Blackwellised particle filtering for dynamic Bayesian networks, Sequential Monte Carlo methods in practice: Springer, 2001: 499-515.
- [15] Beevers K R, Huang W H. SLAM with sparse sensing[C]. Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006., 2006: 2285-2290.
- [16] Brunskill E, Roy N. SLAM using incremental probabilistic PCA and dimensionality reduction[C]. Proceedings of the 2005 IEEE International Conference on Robotics and Automation, 2005: 342-347.
- [17] Montemarlo M. FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem[C]. Proc of Theaaai National Conference on Artificial Intelligence,

- 2002.
- [18] Montemerlo M, Thrun S, Koller D, et al. FastSLAM 2.0: An improved particle filtering algorithm for simultaneous localization and mapping that provably converges[C]. IJCAI, 2003: 1151-1156.
 - [19] Grisetti G, Stachniss C, Burgard W. Improving grid-based slam with rao-blackwellized particle filters by adaptive proposals and selective resampling[C]. Proceedings of the 2005 IEEE international conference on robotics and automation, 2005: 2432-2437.
 - [20] Liao M, Wang D, Yang H. Deploy Indoor 2D Laser SLAM on a Raspberry Pi-Based Mobile Robot[C]. 2019 11th International Conference on Intelligent Human-Machine Systems and Cybernetics (IHMSC), 2019: 7-10.
 - [21] 沈斯杰, 田昕, 魏国亮, 等. 基于 2D 激光雷达的 SLAM 算法研究综述 [J]. 计算机技术与发展, 2022, 32(01): 13-18.
 - [22] 周治国, 曹江微, 邸顺帆. 3D 激光雷达 SLAM 算法综述 [J]. 仪器仪表学报, 2021, 42(09): 13-27.
 - [23] Lu F, Milios E. Globally consistent range scan alignment for environment mapping [J]. Autonomous robots, 1997, 4(4): 333-349.
 - [24] Golfarelli M, Maio D, Rizzi S. Elastic correction of dead-reckoning errors in map building[C]. Proceedings. 1998 IEEE/RSJ International Conference on Intelligent Robots and Systems. Innovations in Theory, Practice and Applications (Cat. No. 98CH36190), 1998: 905-911.
 - [25] Gutmann J-S, Konolige K. Incremental mapping of large cyclic environments[C]. Proceedings 1999 IEEE International Symposium on Computational Intelligence in Robotics and Automation. CIRA'99 (Cat. No. 99EX375), 1999: 318-325.
 - [26] Konolige K, Garage W. Sparse Sparse Bundle Adjustment[C]. BMVC, 2010: 102.1-102.11.
 - [27] Hess W, Kohler D, Rapp H, et al. Real-time loop closure in 2D LIDAR SLAM[C]. 2016 IEEE international conference on robotics and automation (ICRA), 2016: 1271-1278.
 - [28] Zhu J, Xu L. Design and implementation of ROS-based autonomous mobile robot positioning and navigation system[C]. 2019 18th International Symposium on Distributed Computing and Applications for Business Engineering and Science (DCABES), 2019: 214-217.
 - [29] Chen Y, Medioni G. Object modelling by registration of multiple range images [J]. Image and vision computing, 1992, 10(3): 145-155.
 - [30] Minguez J, Montesano L, Lamiroux F. Metric-based iterative closest point scan matching for sensor displacement estimation [J]. IEEE Transactions on Robotics, 2006, 22(5): 1047-1054.
 - [31] Censi A. An ICP variant using a point-to-line metric[C]. 2008 IEEE International Conference on Robotics and Automation, 2008: 19-25.
 - [32] Segal A, Haehnel D, Thrun S. Generalized-icp[C]. Robotics: science and systems, 2009: 435.
 - [33] Hong S, Ko H, Kim J. VICP: Velocity updating iterative closest point algorithm[C]. 2010 IEEE International Conference on Robotics and Automation, 2010: 1893-1898.

- [34] Yang J, Li H, Jia Y. Go-icp: Solving 3d registration efficiently and globally optimally[C]. Proceedings of the IEEE International Conference on Computer Vision, 2013: 1457-1464.
- [35] Jensfelt P, Kristensen S. Active global localization for a mobile robot using multiple hypothesis tracking [J]. IEEE Transactions on Robotics and Automation, 2001, 17(5): 748-760.
- [36] Nakamura T, Wakita S. Robust global scan matching method using congruence transformation invariant feature descriptors and a geometric constraint between keypoints[C]. 2014 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2014: 3103-3108.
- [37] Mohamed H, Moussa A, Elhabiby M, et al. Improved real-time scan matching using corner features [J]. Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci, 2016: 533-539.
- [38] Liu S, Atia M M, Gao Y, et al. Adaptive covariance estimation method for LiDAR-aided multi-sensor integrated navigation systems [J]. Micromachines, 2015, 6(2): 196-215.
- [39] Siadat A, Kaske A, Klausmann S, et al. An optimized segmentation method for a 2D laser-scanner applied to mobile robot navigation [J]. IFAC Proceedings Volumes, 1997, 30(7): 149-154.
- [40] Núñez P, Vázquez-Martín R, Del Toro J, et al. Natural landmark extraction for mobile robot navigation based on an adaptive curvature estimation [J]. Robotics and Autonomous Systems, 2008, 56(3): 247-264.
- [41] Biber P, Straßer W. The normal distributions transform: A new approach to laser scan matching[C]. Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)(Cat. No. 03CH37453), 2003: 2743-2748.
- [42] Kohlbrecher S, Von Stryk O, Meyer J, et al. A flexible and scalable SLAM system with full 3D motion estimation[C]. 2011 IEEE international symposium on safety, security, and rescue robotics, 2011: 155-160.
- [43] Olson E B. Real-time correlative scan matching[C]. 2009 IEEE International Conference on Robotics and Automation, 2009: 4387-4393.
- [44] 文国成, 曾碧, 陈云华. 一种适用于激光 SLAM 大尺度地图的闭环检测方法 [J]. 计算机应用研究, 2018, 35(06): 1724-1727+1732.
- [45] Olson E. M3RSM: Many-to-many multi-resolution scan matching[C]. 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015: 5815-5821.
- [46] Yin H, Ding X, Tang L, et al. Efficient 3D LIDAR based loop closing using deep neural network[C]. 2017 IEEE International Conference on Robotics and Biomimetics (ROBIO), 2017: 481-486.
- [47] Rusu R B, Blodow N, Beetz M. Fast point feature histograms (FPFH) for 3D registration[C]. 2009 IEEE international conference on robotics and automation, 2009: 3212-3217.
- [48] 梁潇. 基于激光与单目视觉融合的机器人室内定位与制图研究[D]. 哈尔滨工业大学, 2016.
- [49] 邹谦. 基于图优化 SLAM 的移动机器人导航方法研究[D]. 哈尔滨工业大学, 2017.
- [50] 徐晓苏, 代维, 杨博, 等. 室内环境下基于图优化的视觉惯性 SLAM 方法 [J]. 中国惯性技术学报, 2017, 25(03): 313-319.

- [51] Frese U, Larsson P, Duckett T. A multilevel relaxation algorithm for simultaneous localization and mapping [J]. *IEEE Transactions on Robotics*, 2005, 21(2): 196-207.
- [52] Dellaert F, Kaess M. Square Root SAM: Simultaneous localization and mapping via square root information smoothing [J]. *The International Journal of Robotics Research*, 2006, 25(12): 1181-1203.
- [53] Kaess M, Ranganathan A, Dellaert F. iSAM: Incremental smoothing and mapping [J]. *IEEE Transactions on Robotics*, 2008, 24(6): 1365-1378.
- [54] Kaess M, Johannsson H, Roberts R, et al. iSAM2: Incremental smoothing and mapping using the Bayes tree [J]. *The International Journal of Robotics Research*, 2012, 31(2): 216-235.
- [55] Olson E, Leonard J, Teller S. Fast iterative alignment of pose graphs with poor initial estimates[C]. *Proceedings 2006 IEEE International Conference on Robotics and Automation*, 2006. ICRA 2006., 2006: 2262-2269.
- [56] Grisetti G, Stachniss C, Grzonka S, et al. A tree parameterization for efficiently computing maximum likelihood maps using gradient descent[C]. *Robotics: Science and Systems*, 2007: 9.
- [57] Gao C, Harle R. MSGD: Scalable back-end for indoor magnetic field-based GraphSLAM[C]. *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017: 3855-3862.
- [58] Duckett T, Marsland S, Shapiro J. Learning globally consistent maps by relaxation[C]. *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065)*, 2000: 3841-3846.
- [59] Carlone L, Censi A, Dellaert F. Selecting good measurements via ℓ_1 relaxation: A convex approach for robust estimation over graphs[C]. *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014: 2667-2674.
- [60] Grisetti G, Kümmerle R, Stachniss C, et al. Hierarchical optimization on manifolds for online 2D and 3D mapping[C]. *2010 IEEE International Conference on Robotics and Automation*, 2010: 273-278.
- [61] Kümmerle R, Grisetti G, Strasdat H, et al. g2o: A general framework for graph optimization[C]. *2011 IEEE International Conference on Robotics and Automation*, 2011: 3607-3613.
- [62] Hertzberg C, Wagner R, Frese U, et al. Integrating generic sensor fusion algorithms with sound state representations through encapsulation of manifolds [J]. *Information Fusion*, 2013, 14(1): 57-77.
- [63] Forster C, Carlone L, Dellaert F, et al. On-manifold preintegration for real-time visual-inertial odometry [J]. *IEEE Transactions on Robotics*, 2016, 33(1): 1-21.
- [64] Fox 著 美塞特 S T 德沃比 W B 美迪福 D. 概率机器人[M]. 北京: 机械工业出版社, 2017: 495.
- [65] Grisetti G, Kümmerle R, Stachniss C, et al. A tutorial on graph-based SLAM [J]. *IEEE Intelligent Transportation Systems Magazine*, 2010, 2(4): 31-43.